TCP4D: A Unified Tool-Center-Point Tracking and Generation Model for Robot Manipulation

Haosong Peng[1*], Xiangyang Liu[2*], Zhengshen Zhang[3], Yunfeng Fan[4],
Tianyu Qi[5], Xin Xia[6], Yufeng Zhan[2✉], Wenchao Xu[1✉]

[1] The Hong Kong University of Science and Technology [2] Beijing Institute of Technology
[3] National University of Singapore [4] The Hong Kong Polytechnic University [5] Sun Yat-sen University
[6] Zhejiang University

Project page: https://livioni.github.io/TCP4D/

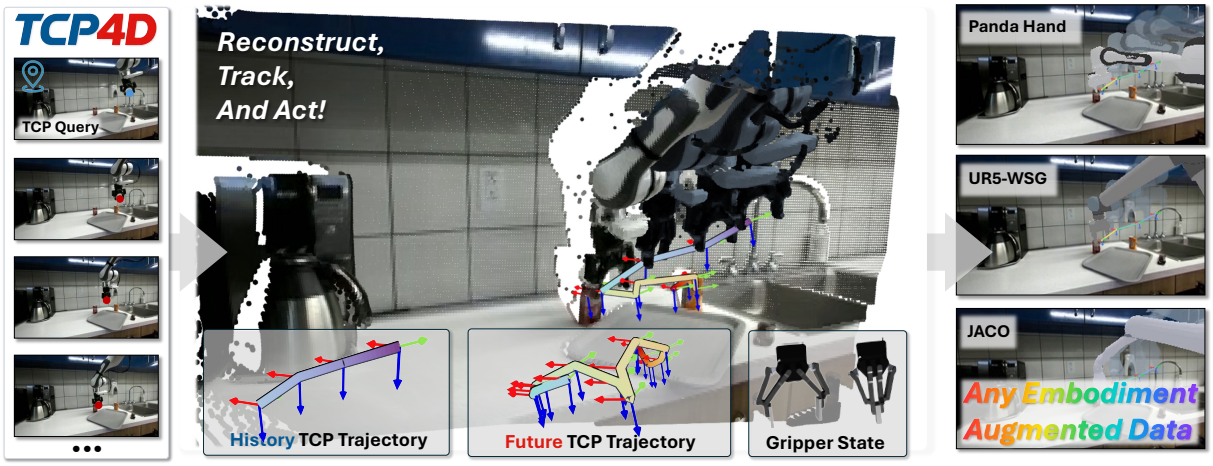

Figure 1: We propose **TCP4D**, a unified model that tracks tool-center-point (TCP) poses and gripper states from RGB videos and generates task-conditioned future TCP trajectories and gripper states, enabling robot manipulation and cross-embodiment data augmentation.

Abstract

Robot manipulation videos provide rich visual experience, yet recovering robot action and gripper-state annotations from RGB observations remains challenging. We present **TCP4D**, a unified model that tracks tool-center-point (TCP) motion and generates future TCP trajectories using a shared 4D scene representation. Given a monocular video and initial 2D TCP queries, the **tracker** recovers metric TCP poses in $\mathrm{SE}(3)$ and gripper states alongside scene geometry and camera parameters. The **generator** samples local features from the same 4D representation along the recovered TCP tracks, then combines them with robot observations and language instructions to predict future TCP states. Our new cross-embodiment augmentation pipeline retargets recovered motion and synthesizes target-robot observations from RGB demonstrations without recorded source-robot states. Experiments show accurate TCP tracking on DROID and RoboLab and effective few-shot adaptation to the new embodiment. The generator achieves 69.19% mean success on RoboTwin and outperforms well-established policy models on two real-world tasks on a physical platform. Joint training on source and augmented demonstrations improves mean task success by 41.5% on a target robot compared with training on augmented demonstrations alone.

*Equal Contribution. ✉Corresponding Authors.

1 Introduction

Large-scale video data of robot manipulation (Open X-Embodiment Collaboration et al., 2023; Contributors et al., 2026) offer a rich source of experience for learning diverse skills. These data provide an important foundation for modern vision-language-action (VLA) models (Pan et al., 2026; Intelligence et al., 2025) and world action models (WAMs) (Ye et al., 2026; Yuan et al., 2026b). However, not all videos include synchronized action annotations, and recovering these labels from RGB video alone remains a nontrivial challenge.

Recovering the end effector's motion offers a practical route to obtaining such supervision from video. This motion can be represented by the tool-center-point (TCP) pose (i.e., $\mathrm{SE}(3)$), which specifies the position and orientation of the end effector's tool frame. With a common reference frame, TCP provides a unified action representation: the same pose encodes the same task-space goal across robot platforms, even when their joint configurations differ (Xiaomi Embodied Intelligence Team et al., 2026; Xie et al., 2026). TCP trajectories express demonstrated motion and future action targets in the same task space, providing a common interface for motion recovery and action generation.

Recent 4D point tracking methods (Luo et al., 2026; Karhade et al., 2026; Lu et al., 2026; Sucar et al., 2026) recover scene geometry and track 3D surface points from RGB video. The TCP, however, is the origin of a virtual tool frame and can lie in free space between the gripper fingers. Its image projection can therefore fall on a background pixel. Moreover, point trackers predict only 3D positions, but not tool orientation, so even an accurate trajectory of the TCP origin does not determine the full TCP pose. Recovering TCP poses from these trackers requires additional information and complex post-processing.

To address this challenge, we propose **TCP4D**, a unified model that uses a shared 4D scene representation to track observed TCP motion and generate future TCP trajectories for robot manipulation. Given a monocular RGB video and an initial 2D TCP query for each arm, the TCP4D *Tracker* directly estimates TCP poses in $\mathrm{SE}(3)$ and gripper states throughout the observed sequence, together with metric scene geometry and camera parameters. The TCP4D *Generator* extracts local visual features along the observed end-effector motion. A diffusion Transformer (DiT) combines these features with the shared scene representation to generate task-conditioned future TCP poses in $\mathrm{SE}(3)$ and gripper states, serving as a manipulation policy.

Building on TCP4D, we introduce a novel cross-embodiment data augmentation pipeline that converts RGB videos into demonstrations for target robots. Unlike existing methods that rely on recorded source states and actions (Chen et al., 2024b; Ji et al., 2025), our pipeline recovers TCP trajectories and gripper states directly from video. The recovered motion is retargeted to alternative embodiments through inverse kinematics (IK), while visual composition produces corresponding target-robot observations. This enables the reuse of videos without source-state annotations to construct demonstrations for cross-embodiment policy learning.

We conduct extensive experiments to validate the effectiveness of TCP4D for TCP tracking and generation, as well as the utility of our cross-embodiment data augmentation pipeline. For tracking, TCP4D achieves absolute trajectory errors of 7.60 and 16.07 mm on DROID and RoboLab, respectively, and adapts to the UR5 with an error of 8.85 mm using just ten demonstrations. As a manipulation policy, its TCP generator achieves 69.19% average success on RoboTwin benchmark under *clean2clean* and 78.33% across two real-world tasks on CoBoT Magic. Training jointly on source and augmented demonstrations raises the mean success rate from 53.5% to 65.5% on the source robot and by 41.5 % and 7.0 % on the two target robots, relative to training on source-only and augmented-only data, respectively. Our main contributions are as follows:

- We introduce TCP4D, a unified model that directly recovers TCP poses and gripper states from RGB videos and generates future TCP trajectories conditioned on observations and instructions. TCP4D adapts effectively to a new robot embodiment with only a few demonstrations.
- We develop a novel cross-embodiment augmentation pipeline that transforms RGB demonstrations into target-robot training data through recovered TCP motion, without requiring recorded source-robot state trajectories.

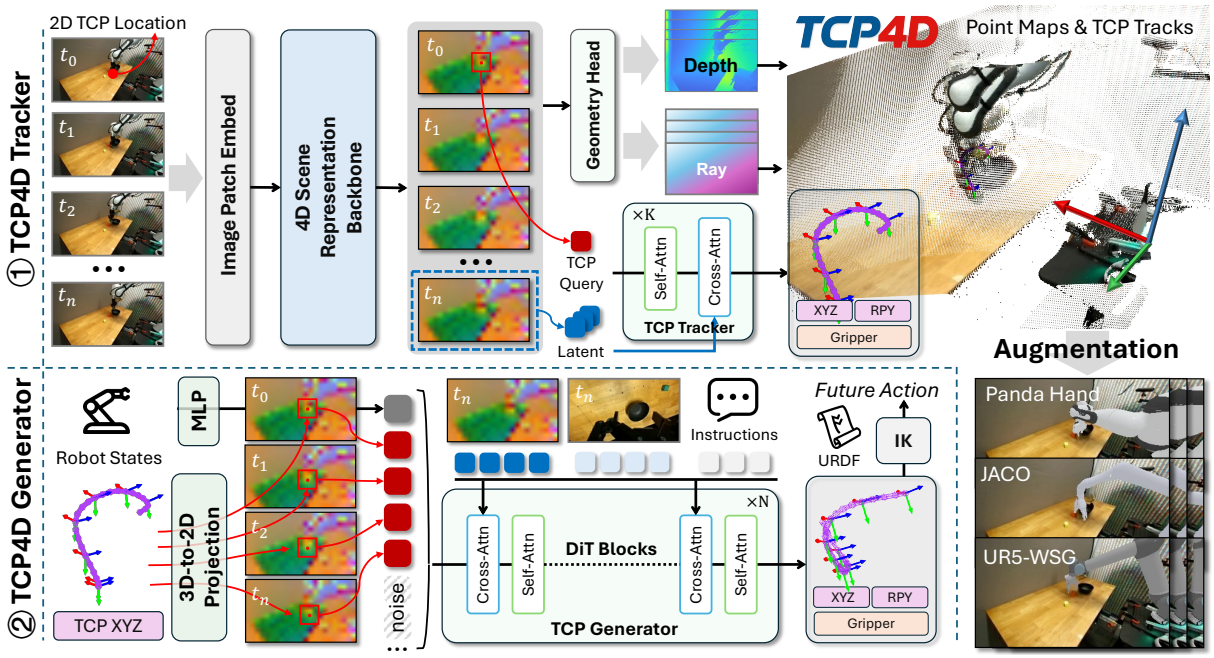

Figure 2: **Overview of TCP4D.** ① TCP4D Tracker takes a monocular video as input and constructs a 4D scene representation. Given the TCP's 2D location in the initial frame as a query, it decodes the scene geometry, the TCP's $\mathrm{SE}(3)$ motion, and the gripper state. ② TCP4D Generator extracts local features along the TCP tracks to form queries and incorporates the robot states. Conditioned on the current 4D scene representation, the wrist RGB view, and language instruction, it predicts the TCP motion and gripper state over the next h steps.

- Extensive experiments across diverse tasks in simulation benchmarks and real-world settings demonstrate the strong performance of TCP4D in TCP tracking and manipulation, as well as the effectiveness of its cross-embodiment data augmentation.

2 Related Work

4D Reconstruction and Point Tracking. Recent feed-forward approaches to 4D reconstruction include dense geometric prediction (Wang et al., 2026b; Peng et al., 2026; Sucar et al., 2026; Lu et al., 2026; Jiang et al., 2026) and query-based frameworks (Zhang et al., 2026a; Luo et al., 2026) that provide flexible access to scene geometry and motion across space and time. Beyond tracking observed motion, recent methods (Zhang et al., 2026b; Duisterhof et al., 2026) predict future point-cloud dynamics through 3D trajectory forecasting and completion. These approaches primarily characterize scene geometry and point positions, leaving local orientation implicit. Concurrent work MoSE3 (Cheng et al., 2026) extends motion estimation to dense per-pixel $\mathrm{SE}(3)$ transforms by fitting rigid motions within soft clusters of tracked scene points. In contrast, TCP4D directly tracks the $\mathrm{SE}(3)$ pose of a virtual TCP, without requiring a corresponding physical surface point or rigid-body grouping.

Point-based Manipulation Policy. Prior work uses predicted 2D point tracks to guide manipulation policies (Bharadhwaj et al., 2024a; Cao et al., 2026; Wen et al., 2024). In 3D, world models predict geometric motion to support planning and action learning (Zhi et al., 2025; Lee et al., 2026; Huang et al., 2026a; Wang et al., 2026d; Hu et al., 2026). Other approaches infer waypoints or poses from visual predictions (Huang et al., 2026b; Zhang et al., 2025), or incorporate motion cues into VLA policies (Wang et al., 2026a; Yuan et al., 2026a). Point-based policies also use depth-derived scene geometry to ground action prediction. DP3 (Ze et al., 2024) conditions action diffusion on compact features from sparse point clouds, while Act3D (Gervet et al., 2023) searches a 3D feature field for the next end-effector position. 3D Diffuser Actor (Ke et al., 2024) denoises pose trajectories conditioned on 3D scene features. TCP4D uses TCP pose and gripper state as a shared interface between motion tracking and action generation, tracking them across observed frames in a 4D scene representation reconstructed from RGB video.

Cross-Embodiment Augmentation Pipeline. Existing approaches to robot data augmentation include simulation-based demonstration synthesis (Mandlekar et al., 2023; Dai et al., 2024) and image-space

semantic augmentation (Yu et al., 2023; Bharadhwaj et al., 2024b), which expand motion and scene diversity without additional real-world collection. For embodiment transfer, Mirage (Chen et al., 2024a) uses test-time cross-painting to present a target robot as the source robot expected by a pretrained policy. RoVi-Aug (Chen et al., 2024b) uses diffusion-based image generation to replace source-robot appearances with target embodiments while retaining the associated task-space motion. OXE-AugE (Ji et al., 2025) further scales embodiment augmentation through simulation-based trajectory retargeting, background inpainting, and target-robot rendering. However, RoVi-Aug and OXE-AugE assume access to recorded source end-effector poses and actions. In contrast, TCP4D recovers TCP trajectories in $\mathrm{SE}(3)$ and gripper states from RGB videos and obtains target joint configurations through IK, enabling augmentation without recorded source-robot states.

3 Methodology

We present TCP4D, a feed-forward framework that tracks and predicts TCP motion in $\mathrm{SE}(3)$ from monocular videos of robot manipulation, as illustrated in Fig. 2. We first formulate the tasks in Sec. 3.1, introduce the TCP4D Tracker and Generator in Sec. 3.2 and 3.3, and describe two-stage training in Sec. 3.4. Finally, Sec. 3.5 introduces a novel downstream data augmentation pipeline using TCP4D.

3.1 Problem Formulation

TCP Tracking. Given an RGB sequence $\mathbf{I} = \{I_i\}_{i=1}^{N}$ and a 2D TCP query $\mathbf{u}_1^a = \langle x_1^a, y_1^a \rangle$ specifying the TCP's pixel coordinates in the reference frame, our goal is to track the TCP state $\mathbf{x}_i^a = (\mathbf{T}_i^a, g_i^a) \in \mathrm{SE}(3) \times [0,1]$ at each observed frame. Here, $\mathbf{T}_i^a = (\mathbf{R}_i^a, \mathbf{t}_i^a) \in \mathrm{SE}(3)$ denotes the TCP pose in frame i's camera coordinate system, comprising orientation $\mathbf{R}_i^a \in \mathrm{SO}(3)$ and position $\mathbf{t}_i^a \in \mathbb{R}^3$; g_i^a denotes gripper state, and $a \in \{l, r\}$ indexes left or right robot arms. TCP4D jointly predicts scene geometry as metric depth maps $\hat{\mathbf{D}}_i \in \mathbb{R}^{H \times W}$ and camera-ray maps $\hat{\mathbf{M}}_i \in \mathbb{R}^{\frac{H}{2} \times \frac{W}{2} \times 6}$ encoding camera intrinsics and extrinsics relative to the first camera frame through per-pixel ray directions and origins. We write the TCP4D tracker $\mathcal{T}_\theta$ as:

$$\left\{\{\hat{\mathbf{x}}_i^a\}, \hat{\mathbf{D}}_i, \hat{\mathbf{M}}_i\right\}_{i=1}^{N} = \mathcal{T}_\theta(\mathbf{I}, \{\mathbf{u}_1^a\}). \tag{1}$$

TCP Generation. Given a language instruction ℓ, our goal is to generate the future TCP states over an h-step horizon conditioned on the available observations at time t. These observations comprise L past and current RGB frames $\{I_i\}_{i=t-L+1}^{t}$, the current joint and gripper state $\mathbf{s}_t$, and current wrist-camera RGB observations $I_t^{w,a}$. We collect these observations as $\mathcal{O}_t = (\{I_i\}_{i=t-L+1}^{t}, \mathbf{s}_t, \{I_t^{w,a}\})$ and write the language-conditioned generator $\mathcal{G}_\theta$ as

$$\hat{\mathbf{A}}_t = \mathcal{G}_\theta(\boldsymbol{\epsilon}; \mathcal{O}_t, \ell) = \left\{\hat{\mathbf{x}}_{t+k}^a\right\}_{k=1,\ldots,h}, \tag{2}$$

where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is the initial noise for trajectory generation. All future TCP poses are expressed in the camera coordinate system.

3.2 TCP4D Tracker

4D Scene Representation Encoding. Following 4RC (Luo et al., 2026), we encode the input video into a unified 4D scene representation using a DINOv2-based Transformer (Oquab et al., 2024). Each frame is divided into patch tokens and augmented with a learnable camera token and a time token. The backbone alternates frame-wise self-attention with global self-attention across frames to capture spatial structure and temporal correspondences. We write

$$\mathcal{F} = \mathcal{E}_\theta(\mathbf{I}) = \left\{(\mathbf{F}_i, \mathbf{c}_i, \mathbf{e}_i)\right\}_{i=1}^{N}, \tag{3}$$

where $\mathbf{F}_i$ contains the encoded 4D representation tokens, $\mathbf{c}_i$ is the camera token summarizing camera geometry, and $\mathbf{e}_i$ is the time token aggregating temporal context for frame i.

Geometry Head. A geometry head with a Dual-DPT (Lin et al., 2026) decoder maps the encoded 4D representation tokens to metric depth maps $\hat{\mathbf{D}}_i \in \mathbb{R}^{H \times W}$ and camera-ray maps $\hat{\mathbf{M}}_i \in \mathbb{R}^{\frac{H}{2} \times \frac{W}{2} \times 6}$, along

with their confidence maps. A lightweight camera head additionally decodes $\mathbf{c}_i$ into camera pose relative to the robot base and field of view.

Sparse TCP Queries. To identify each arm's TCP, a visual query encoder $\mathcal{Q}_\theta$ bilinearly samples a 3×3 neighborhood from $\mathbf{F}_1$ around $\mathbf{u}_1^a$. A residual adapter and learned arm and offset embeddings encode local appearance and query identity. We extract these sparse query tokens at four backbone levels and repeat them across target frames. We first encode TCP queries in the reference frame and then decode their representations at each target frame:

$$\mathbf{Q}_1 = \mathcal{Q}_\theta\big(\mathbf{F}_1, \{\mathbf{u}_1^a\}_a\big), \quad \mathbf{Z}_i = \mathcal{D}_\theta\big(\mathbf{Q}_1, \mathbf{F}_i; \mathbf{e}_i\big), \tag{4}$$

where $\mathbf{Q}_1$ contains the sparse TCP query tokens for all arms and $\mathbf{Z}_i$ contains their decoded representations at frame i. Each layer first applies cross-attention, with the query sequence providing queries and $\mathbf{F}_i$ providing keys and values, and then applies self-attention within the query sequence. Reference-frame sampling positions and target-frame patch positions enter the attention operations through 2D RoPE (Heo et al., 2024). The time token $\mathbf{e}_i$ additionally modulates the self-attention and feed-forward layers through AdaLN (Perez et al., 2018). Finally, a lightweight prediction head regresses TCP translation $\hat{\mathbf{t}}_i^a$, a continuous 6D rotation representation (Zhou et al., 2019), gripper state $\hat{g}_i^a$, and pose confidence. The rotation prediction is projected to $\mathrm{SO}(3)$ to form $\hat{\mathbf{T}}_i^a$. Together with $\hat{g}_i^a$, this yields the tracked state $\hat{\mathbf{x}}_i^a$.

3.3 TCP4D Generator

Tracking-conditioned Representation. The Generator builds on the 4D scene feature $\mathbf{F}_i$ and TCP tracks $\{\hat{\mathbf{x}}_i^a\}_{i=t-L+1}^t$. We project each tracked TCP translation $\hat{\mathbf{t}}_i^a$ into its observed frame using the predicted camera intrinsics to obtain image queries $\hat{\mathbf{u}}_i^a$, and extract local TCP tokens with the shared encoder $\mathcal{Q}_\theta(\mathbf{F}_i, \{\hat{\mathbf{u}}_i^a\}_a)$. We add spatial and temporal embeddings to the local TCP tokens to encode their image locations and observation order.

Conditional Motion Generation. A DiT takes a single sequence formed by concatenating an embedding of the robot state $\mathbf{s}_t$, the local TCP tokens, and noisy future-action tokens. Self-attention jointly updates this sequence, while cross-attention uses current scene features $\mathbf{F}_t$, wrist-image features, and language features as keys and values. A shared frozen DINOv2 encodes the wrist images $I_t^{w,a}$, and a frozen T5-base encoder (Raffel et al., 2020) encodes the instruction ℓ. We train the DiT with flow matching (Lipman et al., 2023). For a normalized target action sequence $\mathbf{A}$, the noisy input and target velocity are

$$\mathbf{A}_\tau = (1-\tau)\boldsymbol{\epsilon} + \tau\mathbf{A}, \qquad \mathbf{v}^\star = \mathbf{A} - \boldsymbol{\epsilon}, \tag{5}$$

where $\tau \in [0, 1)$ is flow time. The output head maps the future-action tokens to predicted velocities $\mathbf{v}_\phi(\mathbf{A}_\tau, \tau; \mathcal{O}_t, \ell)$. At inference, four Euler updates transform $\boldsymbol{\epsilon}$ into the future action sequence, which is denormalized and decoded into $\hat{\mathbf{A}}_t$.

3.4 Training Objectives

We train TCP4D in two stages with different objectives and training strategies. Loss definitions, normalization, and component weights are provided in Appx. A.

Stage 1: Geometry and TCP Reconstruction. We jointly train the visual backbone, geometry head, and TCP Tracker to reconstruct metric scene geometry and TCP states for the available arms:

$$\mathcal{L}_{\text{stage1}} = \mathcal{L}_{\text{geo}} + \lambda_{\text{tcp}}\mathcal{L}_{\text{tcp}}^{(1)}. \tag{6}$$

The geometry loss $\mathcal{L}_{\text{geo}}$ combines confidence-weighted depth and ray regression with a multiscale depth-gradient term. TCP supervision $\mathcal{L}_{\text{tcp}}^{(1)}$ combines a confidence-weighted rigid-point pose error, rotational alignment, temporal velocity consistency, and gripper prediction.

Stage 2: Conditional TCP Generation. We initialize the backbone and Tracker modules from Stage 1 and jointly fine-tune them with the conditioning modules and DiT using:

$$\mathcal{L}_{\text{stage2}} = \mathcal{L}_{\text{geo}} + \lambda_{\text{tcp}}\mathcal{L}_{\text{tcp}}^{(2)} + \lambda_{\text{fm}}\mathcal{L}_{\text{fm}}. \tag{7}$$

The flow-matching loss $\mathcal{L}_{\text{fm}}$ supervises the predicted velocities of position, rotation, and gripper components for both arms. $\mathcal{L}_{\text{tcp}}^{(2)}$ retains pose and gripper supervision and omits the temporal velocity term.

3.5 Cross-Embodiment Augmentation Pipeline

As shown in Fig. 3, we use TCP4D Tracker to recover TCP motion and the gripper state from source demonstration videos and retarget them to alternative embodiments. We transform the predicted camera-frame poses into a common world frame, align TCP conventions, and solve target-robot inverse kinematics with fixed bases while preserving the world-frame trajectories. Source-robot masks are obtained either with SAM3 (Carion et al., 2026) or by projecting the source URDF using recorded robot states. We erase the masked robot, complete the background with a video inpainting model (Zhou et al., 2023), and composite the rendered target robot into the completed scene. Depth is optional and can refine robot masks when available. For example, when the projected robot model overlaps with other scene elements in the image, depth information helps determine which is in front and refine the robot mask accordingly. Fig. 5 shows an example of our cross-embodiment augmented data. Further implementation and export details are provided in Appx. B.

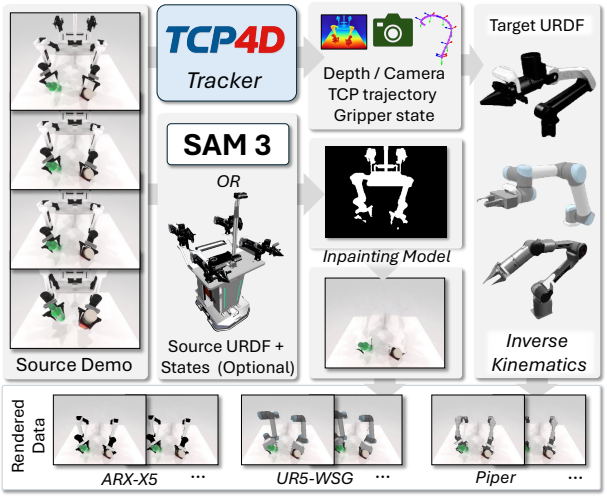

Figure 3: **Cross-embodiment augmentation pipeline.** Given a video, the TCP4D tracker reconstructs the TCP trajectory and gripper state. We remove and inpaint the original robot region, then retarget the motion to alternative embodiments using IK.

4 Experiments

We evaluate TCP4D across four dimensions: (1) TCP tracking performance, (2) the effectiveness of cross-embodiment data augmentation, (3) robot manipulation performance of the TCP generator, and (4) real-world manipulation performance on a physical robot platform.

4.1 Training Setup

Datasets and Benchmark. We use the subset of DROID (Khazatsky et al., 2024) annotated by PointWorld (Huang et al., 2026a), comprising 33,966 episodes (67,932 camera sequences) of Franka Panda 7DOF Arm. We use 90% of the episodes for training and randomly sample 100 episodes from the remaining 10% as the evaluation set. For in-the-wild evaluation, we additionally collect 100 episodes spanning 100 distinct tasks in RoboLab (Yang et al., 2026b). To evaluate robot manipulation, we use RoboTwin (Chen et al., 2025) and collect 50 clean demonstrations per task for 48 tasks, using the third-person view. We train on these demonstrations and evaluate under both *clean2clean* and *clean2random* settings.

Implementation Details. In Stage 1, we train two separate TCP trackers on DROID and RoboTwin, respectively, initializing their visual backbones and geometry modules from pretrained 4RC (Luo et al., 2026) weights. RGB image resolutions are 320×180 for DROID and 320×240 for RoboTwin. We sample Stage 1 clips with random starting frames, a per-clip temporal stride randomly chosen from 1–5, and temporal reversal. In Stage 2, we use only RoboTwin data to train the TCP generator. Each sample contains $L = 4$ consecutive observation frames paired with the next $h = 32$ TCP states, preserving chronological order. The Stage 1 tracker and the full TCP4D model contain approximately 1.522B and 2.272B parameters, respectively, including frozen modules and retained heads. Further implementation details are provided in Appx. C.

4.2 TCP4D Tracking Performance

Baselines. We compare TCP4D with VGGT-Omega (Wang et al., 2026b), DA3 (Lin et al., 2026), MapAnything (Keetha et al., 2026), 4RC (Luo et al., 2026), VDPM (Sucar et al., 2026), Any4D (Karhade et al., 2026), Track4World (Lu et al., 2026), CoTracker3 (Karaev et al., 2025), and MoSE3 (Cheng et al.,

Table 1: **TCP4D Tracker evaluation on DROID and Robolab.** All results are camera-wise macro averages. Depth is evaluated within 3 m with per-frame median alignment. $\mathrm{SE}(3)$ errors use Sim(3) alignment. CoTracker3 uses VGGT-Ω depth. The top-2 results are highlighted as best and second, respectively. The metric definitions are given in Appx. D.

DROID (Eval set)

Method	Depth			TCP Tracking		$\mathrm{SE}(3)$			Gripper	
	AbsRel ↓ –	MAE ↓ (mm)	δ_1 ↑ (%)	APD ↑ (%)	MPE ↓ (mm)	ATE ↓ (mm)	RRE ↓ (°)	RTE ↓ (mm)	Acc. ↑ (%)	SR ↑ (%)
VGGT-Ω (Wang et al., 2026b)	0.2279	224.10	66.19	-	-	-	-	-	-	-
DA3-GIANT (Lin et al., 2026)	0.2271	224.35	64.98	-	-	-	-	-	-	-
MapAnything (Keetha et al., 2026)	0.1994	187.42	71.90	-	-	-	-	-	-	-
4RC* (Luo et al., 2026)	0.2272	215.86	68.03	8.74	107.56	84.63	8.37	13.29	-	79.56
Track4World* (Lu et al., 2026)	0.1870	182.56	72.74	3.68	186.95	92.55	10.35	12.42	-	66.05
Any4D* (Karhade et al., 2026)	0.2132	206.72	69.29	2.32	242.01	96.71	10.76	13.10	-	56.09
VDPM (Sucar et al., 2026)*	0.2328	220.05	70.26	4.67	191.06	85.40	8.05	11.58	-	76.82
CoTracker3* (Karaev et al., 2025)	-	-	-	14.04	91.76	70.35	16.14	22.60	-	63.23
MoSE3† (Cheng et al., 2026)	0.1619	156.23	79.16	12.38	103.43	67.65	1.27	5.96	-	89.64
TCP4D	0.0330	31.27	97.74	74.78	11.85	7.60	1.03	3.40	95.84	100.00

RoboLab (Unseen)

Method	Depth			TCP Tracking		$\mathrm{SE}(3)$			Gripper	
	AbsRel ↓ –	MAE ↓ (mm)	δ_1 ↑ (%)	APD ↑ (%)	MPE ↓ (mm)	ATE ↓ (mm)	RRE ↓ (°)	RTE ↓ (mm)	Acc. ↑ (%)	SR ↑ (%)
VGGT-Ω (Wang et al., 2026b)	0.1742	114.29	74.77	-	-	-	-	-	-	-
DA3-GIANT (Lin et al., 2026)	0.1625	104.68	78.93	-	-	-	-	-	-	-
MapAnything (Keetha et al., 2026)	0.1426	93.13	83.34	-	-	-	-	-	-	-
4RC* (Luo et al., 2026)	0.1795	108.26	79.12	4.92	164.53	91.37	7.16	8.59	-	90.62
Track4World* (Lu et al., 2026)	0.1849	112.01	73.57	2.44	208.05	95.43	9.50	8.88	-	82.85
Any4D* (Karhade et al., 2026)	0.1507	95.56	83.89	1.15	238.02	106.49	8.41	8.42	-	74.63
VDPM (Sucar et al., 2026)*	0.1690	108.93	81.31	9.90	111.90	70.90	5.78	8.59	-	96.33
CoTracker3* (Karaev et al., 2025)	-	-	-	8.78	131.39	85.12	15.80	17.63	-	75.38
MoSE3† (Cheng et al., 2026)	0.1142	67.23	89.37	8.58	130.18	85.45	0.99	4.65	-	94.18
TCP4D	0.1718	92.61	92.17	43.71	28.42	16.07	6.80	4.83	81.92	100.00

* TCP poses are recovered by rigid fitting of point tracks. † MoSE3 uses its native dense-point $\mathrm{SE}(3)$ outputs followed by post-processing.

2026) on DROID and RoboLab videos. Since no existing method directly tracks the TCP, we post-process the point tracks predicted by point-tracking baselines to recover the TCP trajectory and pose.

Evaluation Metrics. We report AbsRel, MAE, and δ_1 for depth within 3 m; *Average Percentage of Points* (APD) and *Mean Position Error* (MPE) for 3D TCP tracking; *Absolute Trajectory Error* (ATE), *Relative Rotation Error* (RRE), and *Relative Translation Error* (RTE) for $\mathrm{SE}(3)$ tracking; and tracking *Success Rate* (SR). Gripper Accuracy (Acc.) uses a 0.5 threshold. Metric definitions and evaluation protocols are provided in Appx. D.

Results. As shown in Table 1, TCP4D achieves state-of-the-art performance on most evaluated metrics across both benchmarks. On DROID and RoboLab, it achieves APD scores of 74.78% and 43.71%, respectively, exceeding four times the strongest baseline scores, and reduces ATE to 7.60 and 16.07 mm. TCP4D also achieves 100% SR on both datasets, successfully processing all evaluated tracking windows, compared with the best baseline SRs of 89.64% and 96.33%, respectively. Competing point-tracking methods rely on tracking rigid point groups and fitting rigid transformations in post-processing to recover TCP poses; inaccurate or inconsistent point tracks can yield unreliable pose estimates or prevent successful recovery. By directly predicting TCP poses and gripper states within a unified feed-forward framework, TCP4D avoids this recovery step, delivering accurate and reliable tracking even in unseen manipulation scenarios. Fig. 4 visually compares TCP4D with other baselines. More results can be found in Appx. D.8.

Few-Shot Cross-Embodiment Adaptation. TCP4D demonstrates strong few-shot cross-embodiment

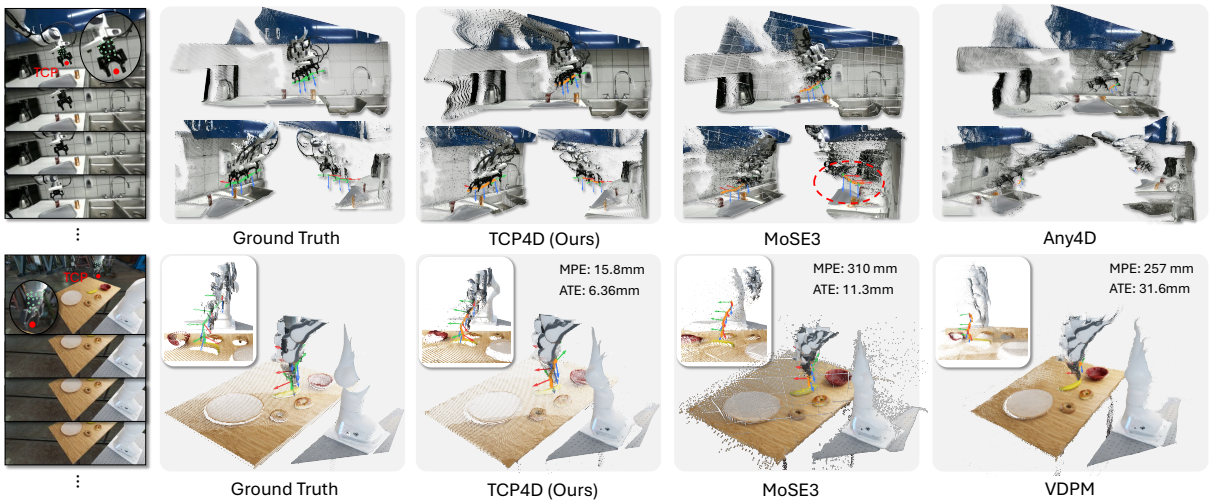

Figure 4: **Qualitative comparison of TCP tracking.** We compare TCP tracking by TCP4D against dense tracking baselines followed by post-processing. The left side shows the input video. The red point in the first frame marks the query used by TCP4D, while the green points indicate the rigid point group tracked by the baselines. More visualizations are provided in Appx. D.8.

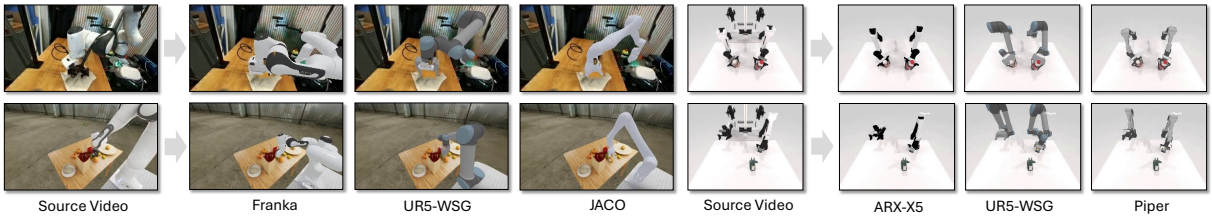

Figure 5: **Example of our cross-embodiment augmented data.** Given an RGB video, TCP4D reconstructs the TCP's 3D trajectory and gripper state, enabling augmentation on any embodiment that satisfies *reachability*. More visualizations in Appx. B.

adaptation capabilities after pretraining on the DROID dataset. We evaluate it on the *Berkeley AUTOLAB UR5* dataset (Chen et al.), which contains 1,000 demonstration episodes using UR5-WSG across different tasks, comprising 97,939 video frames in total. We fine-tune TCP4D under two settings: *(1) three-shot* transfer using *three* randomly selected episodes and *(2) few-shot* transfer using *ten* randomly selected episodes.

As shown in Table 2, *three-shot* adaptation improves APD from 14.39% for MoSE3 to 48.27% and reduces ATE from 44.56 to 12.22 mm. With ten episodes, APD further increases to 64.28%, ATE decreases to 8.85 mm, and gripper accuracy reaches 98.48%. These results demonstrate that TCP4D Tracker can be easily transferred to new environments and robot embodiments using only a small amount of data. The visualizations are provided in Appx. D.9.

Table 2: **TCP4D few shot cross-embodiment adaptation on Berkeley AUTOLAB UR5.** Results are averages over the 100-episode evaluation set.

Method	TCP Tracking		SE(3)			Gripper
	APD ↑ (%)	MPE ↓ (mm)	ATE ↓ (mm)	RRE ↓ (°)	RTE ↓ (mm)	Acc. ↑ (%)
MoSE3†	14.39	88.07	44.56	1.17	6.45	–
Three-Shot	48.27	18.48	12.22	1.03	4.28	97.05
Few-Shot	**64.28**	**11.08**	**8.85**	**0.96**	**3.87**	**98.48**

4.3 Effectiveness of Cross-embodiment Data

Setup. We evaluate whether cross-embodiment demonstrations generated using our pipeline benefit downstream policy learning on four tasks. For each task, we collect 50 source demonstrations on Aloha-AgileX and augment them to ARX-X5 and UR5-WSG, yielding 199 and 198 target demonstrations across the four tasks, respectively. We use DP (Chi et al., 2023) under three training-data settings: *(a) source-only*,

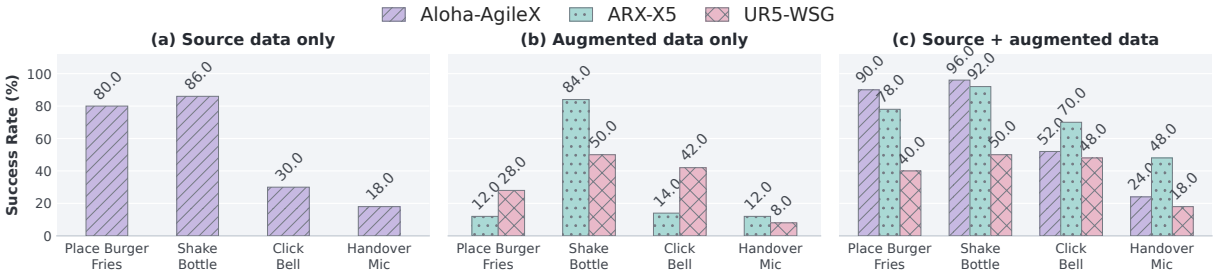

Figure 6: **Cross-embodiment policy evaluation.** Task success rates under three training-data settings on three embodiments. For each task, we collect 50 source demonstrations on Aloha-AgileX and augment them to ARX-X5 and UR5-WSG using our pipeline.

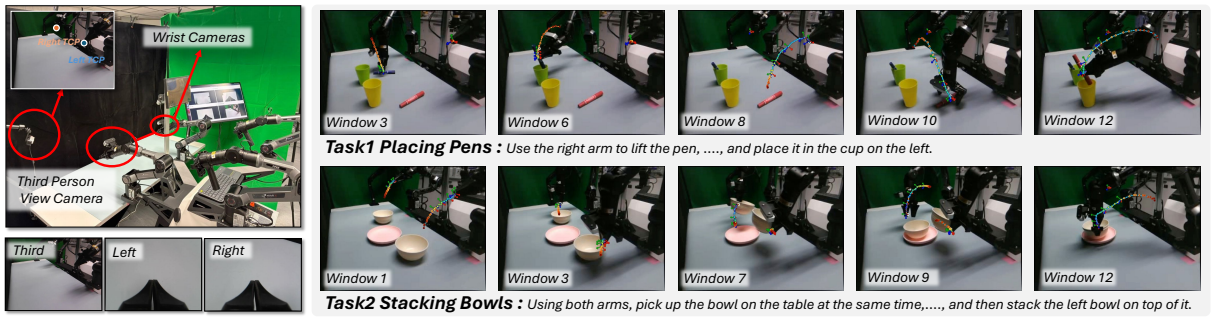

Figure 7: **Real-world experiments setup and task rollouts.** The right panel shows the rollout of tasks *1) placing pens* and *2) stacking bowls*, with the predicted TCP trajectory over the next 32 steps displayed in each inference window. The color gradient from blue to orange indicates the temporal order of future actions. Complete rollout visualizations are in Appx. F.2.

trained on Aloha-AgileX; *(b) augmented-only*, trained separately on each target robot; and *(c) source + augmented*, jointly trained on all three robots and evaluated on each. Policies are trained separately for each task under same budget. Each evaluated policy–embodiment pair is tested over 50 episodes per task. Success rates are reported in Fig. 6, with implementation details and statistics in Appx. E.

Results. As shown in Fig. 6, combining source and augmented demonstrations improves Aloha performance on all four tasks, raising its mean success rate from 53.5% to 65.5%. *Click Bell* shows the largest improvement, from 30% to 52%, demonstrating the effective of augmented demonstrations for policy learning on the source embodiment. On ARX-X5, joint training increases the mean success rate from 30.5% to 72.0% relative to augmented-only training. The gains span all four tasks, including *Place Burger Fries* (12% $\rightarrow$ 78%) and *Click Bell* (14% $\rightarrow$ 70%). On UR5-WSG, joint training improves the mean success rate from 32.0% to 39.0%, with gains on *Place Burger Fries* (28% to 40%), *Click Bell* (42% $\rightarrow$ 48%), and *Handover Mic* (8% $\rightarrow$ 18%). These results show that our pipeline enables policy learning on target robots without original demonstrations from those embodiments, while also improving performance when augmented demonstrations are combined with existing data, highlighting the utility of TCP4D for cross-embodiment data augmentation.

4.4 TCP4D Manipulation Performance

Results. We evaluate TCP4D on 48 RoboTwin tasks with 100 trials per task and report the success rates in Table 3. TCP4D achieves the highest reported average success rate of 69.19% under *clean2clean*. It surpasses established manipulation policies ACT, RDT, and DP3 by 39.94%, 35.27%, and 14.92% , respectively, and also outperforms several VLA models, including EventVLA (65.71%), GalaxeaVLA (62.73%), and π_0 (45.21%). These results support the effectiveness of predicting future TCP motion from 4D scene representations and highlight the potential of this paradigm for robotic manipulation. Complete results for all 48 tasks are provided in Appx. F.1.

The average success rate drops to 10.94% under *clean2random*, where environmental randomization introduces a distribution shift. Our TCP generator is trained solely on *clean* demonstrations. Incorporating large embodiment data pretraining or 3D geometric data from a broader mix of sources could improve

generalization to randomized environments, a direction we leave for future work.

Table 3: **Average success rates (%) on RoboTwin 2.0 over 48 tasks.** "View" indicates the primary input camera view used by each method. The top-2 results in each column are highlighted as best and second, respectively.

Type	Method	View	Easy (C2C)	Hard (C2R)
VLA&WAM	EventVLA (Yang et al., 2026a)	head	65.71	14.50
	AHA-WAM (Cai et al., 2026a)	head	64.38	2.77
	Xiaomi Robotics-0 (Cai et al., 2026b)	head	62.79	17.85
	GalaxeaVLA (Liu et al., 2026)	head	62.73	12.31
	Discrete Forcing (Wang et al., 2026c)	head	59.25	11.04
	StarVLA (Community, 2026)	head	46.73	3.21
	π_0 (Black et al., 2026)	head	45.21	16.15
Policy	DP3 (Ze et al., 2024)	head	54.27	4.04
	RDT (Liu et al., 2025)	head	33.92	13.25
	ACT (Zhao et al., 2023)	head	29.25	1.79
	DP (Chi et al., 2023)	head	27.31	0.67
	TCP4D (Ours)	third	69.19	10.94

4.5 Real-World Evaluation

Real-world setup. We conduct real-world experiments on the CoBoT Magic platform. We evaluate two tasks: *1) placing pens* and *2) stacking bowls* with the left and right arms. Both tasks require large TCP displacements. We collect 60 demonstration trajectories per task for training. We use cuRobo (Sundaralingam et al., 2026) library to solve IK.

Evaluation protocol. We compare TCP4D with two established policies, DP3 (Ze et al., 2024) and ACT (Zhao et al., 2023), using the same demonstration data for fine-tuning and the same evaluation protocol for all methods. For each task and method, we conduct 30 trials and report binary task success and a stage-wise progress score. The task-specific progress score counts the number of completed stages in a predefined task sequence, up to a maximum of four, and its rubric is summarized in Appx. F.2.

Results. As shown in Fig. 8, TCP4D outperforms DP3 and ACT on both real-world tasks in success rate and progress score. It achieves success rates of 83.33% on *placing pens* and 73.33% on *stacking bowls*. Averaged across the two tasks, TCP4D attains a success rate of 78.33%, compared with 63.33% for DP3 and 31.67% for ACT, yielding gains of 15.00 and 46.67 percentage points, respectively.

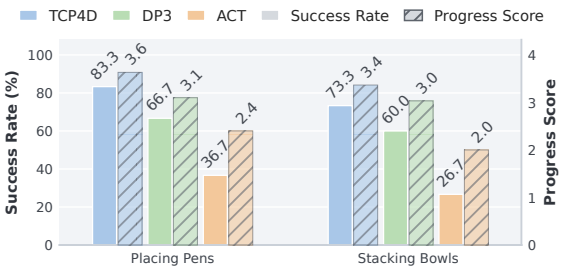

Figure 8: **Real-world task performance.** Solid bars report success rates using the left axis, while hatched bars report average progress scores using the right axis.

Its mean progress score is 3.50 out of 4, indicating that it completes more task stages across the trials. DP3 conditions its policy on sparse 3D point clouds reconstructed from depth observations. In contrast, TCP4D infers scene geometry and TCP motion directly from RGB videos, achieving higher task success without requiring measured depth at inference. These results demonstrate the effectiveness of our 4D scene geometric representation for manipulation while reducing reliance on depth sensing at deployment.

5 Conclusion

We presented TCP4D, a unified model using a shared 4D scene representation to track TCP motion from RGB video and generate future actions conditioned on the recovered tracks. Our cross-embodiment pipeline converts videos into target-robot demonstrations. Experiments demonstrate accurate tracking, few-shot adaptation, effective manipulation, and improved policy learning with augmented data. TCP4D has the potential to scale further with more diverse data and robot embodiments.

References

Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. 2024a. Track2act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part LXXVI*, volume 15134 of *Lecture Notes in Computer Science*, pages 306–324. Springer.

Homanga Bharadhwaj, Jay Vakil, Mohit Sharma, Abhinav Gupta, Shubham Tulsiani, and Vikash Kumar. 2024b. RoboAgent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4788–4795. IEEE.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, and 5 others. 2026. π_0: A vision-language-action flow model for general robot control. *Preprint*, arXiv:2410.24164.

Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, Ru Ying, Ran Zheng, and Yao Mu. 2026a. AHA-WAM:Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *Preprint*, arXiv:2606.09811.

Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, Feng Qiu, Heng Qu, Yifei Su, Qiao Sun, Dong Wang, Donghao Wang, Yunhong Wang, Rujie Wu, Diyun Xiang, and 4 others. 2026b. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *Preprint*, arXiv:2602.12684.

Zhi Cao, Howard Ji, Kevin Zhang, Kuangzhi Ge, Li Fei-Fei, Jiajun Wu, and Huang Huang. 2026. Tract: Bridging robot control and visual prediction with visual tracks. *arXiv preprint arXiv:2608.24101*.

Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris Coll-Vinent, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, and 19 others. 2026. SAM 3: Segment anything with concepts. In *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc.

Lawrence Yunliang Chen, Simeon Adebola, and Ken Goldberg. Berkeley UR5 demonstration dataset. https://sites.google.com/view/berkeley-ur5/home.

Lawrence Yunliang Chen, Karthik Dharmarajan, Kush Hari, Chenfeng Xu, Quan Vuong, and Ken Goldberg. 2024a. MIRAGE: cross-embodiment zero-shot policy transfer with cross-painting. In *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*.

Lawrence Yunliang Chen, Chenfeng Xu, Karthik Dharmarajan, Richard Cheng, Kurt Keutzer, Masayoshi Tomizuka, Quan Vuong, and Ken Goldberg. 2024b. Rovi-aug: Robot and viewpoint augmentation for cross-embodiment robot learning. In *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pages 209–233. PMLR.

Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, and 1 others. 2025. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*.

Jiahuan Cheng, Zhiyi Li, Tian Xia, Ruojin Cai, Yilun Du, and Qianqian Wang. 2026. MoSE3: Learning world-space SE (3) at every pixel. *arXiv preprint arXiv:2610.03716*.

Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. 2023. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems XIX, Daegu, Republic of Korea, July 10-14, 2023*.

StarVLA Community. 2026. StarVLA: A lego-like codebase for vision-language-action model developing. *Preprint*, arXiv:2604.05014.

AgiBot World Contributors, Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, Shu Jiang, Yuxin Jiang, Cheng Jing, Hongyang Li, Jialu Li, Chiming Liu, Yi Liu, Yuxiang Lu, and 33 others. 2026. AgiBot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *Preprint*, arXiv:2503.06669.

Tianyuan Dai, Josiah Wong, Yunfan Jiang, Chen Wang, Cem Gokmen, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. 2024. Automated creation of digital cousins for robust policy learning. In *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pages 4912–4943. PMLR.

Bardienus P Duisterhof, Kaifeng Zhang, Adam Hung, Bowen Wen, Stan Birchfield, Yunzhu Li, Deva Ramanan, and Jeffrey Ichnowski. 2026. PointZero: 3D point track completion for learning transferable 3D dynamics. *arXiv preprint arXiv:2609.19142*.

Théophile Gervet, Zhou Xian, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 2023. Act3d: 3d feature field transformers for multi-task robotic manipulation. In *Conference on Robot Learning, CoRL 2023, 6-9 November 2023, Atlanta, GA, USA*, volume 229 of *Proceedings of Machine Learning Research*, pages 3949–3965. PMLR.

Byeongho Heo, Song Park, Dongyoon Han, and Sangdoo Yun. 2024. Rotary position embedding for vision transformer. In *European Conference on Computer Vision (ECCV)*, pages 289–305, Cham. Springer Nature Switzerland.

Xu Hu, Changjie Wu, Linzhe Shi, Hang Zhang, Chen Yang, Lingjun Zhang, Guowen Zhang, Mu Xu, Junran Peng, Qing Li, and Zhaoxiang Zhang. 2026. Trace-WAM: Learning to act via 3D motion prediction for world action models. *Preprint*, arXiv:2610.36840.

Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. 2026a. Pointworld: Scaling 3d world models for in-the-wild robotic manipulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 20765–20779. Computer Vision Foundation.

Ziqin Huang, Yingyue Li, Chenyangguang Zhang, Ruida Zhang, Yuxin Chen, Gu Wang, Xingyu Liu, Masayoshi Tomizuka, and Xiangyang Ji. 2026b. 3DWay: Generalizing robot manipulation via 3D consistent waypoints. In *European Conference on Computer Vision (ECCV)*, pages 20–38. Springer, Springer Nature Switzerland.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, and 17 others. 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *Preprint*, arXiv:2504.16054.

Guanhua Ji, Harsha Polavaram, Lawrence Yunliang Chen, Sandeep Bajamahal, Zehan Ma, Simeon Adebola, Chenfeng Xu, and Ken Goldberg. 2025. OXE-AugE: A large-scale robot augmentation of OXE for scaling cross-embodiment policy learning. *Preprint*, arXiv:2512.13100.

Yanqin Jiang, Tengfei Wang, Zhengwei Wang, Chenjie Cao, Junta Wu, Wenhan Luo, Weiming Hu, Jin Gao, and Chunchao Guo. 2026. OmniX: Any-view and any-time 4D reconstruction via feed-forward trajectory fields. In *European Conference on Computer Vision (ECCV)*, pages 313–331. Springer, Springer Nature Switzerland.

Nikita Karaev, Yuri Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. 2025. CoTracker3: Simpler and better point tracking by pseudo-labeling real videos. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 1–10. IEEE.

Jay Karhade, Nikhil Varma Keetha, Yuchen Zhang, Tanisha Gupta, Akash Sharma, Sebastian A. Scherer, and Deva Ramanan. 2026. Any4d: Unified feed-forward metric 4d reconstruction. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 14578–14589. Computer Vision Foundation.

Tsung-Wei Ke, Nikolaos Gkanatsios, and Katerina Fragkiadaki. 2024. 3d diffuser actor: Policy diffusion with 3d scene representations. In *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pages 1949–1974. PMLR.

Nikhil Varma Keetha, Norman Müller, Johannes L. Schönberger, Lorenzo Porzi, Yuchen Zhang, Tobias Fischer, Arno Knapitsch, Duncan Zauss, Ethan Weber, Nelson Antunes, Jonathon Luiten, Manuel Lopez-Antequera, Samuel Rota Bulò, Christian Richardt, Deva Ramanan, Sebastian A. Scherer, and Peter Kontschieder. 2026. Mapanything: Universal feed-forward metric 3d reconstruction; map-anything.github.io. In *International Conference on 3D Visio, 3DV 2026, Vancouver, BC, Canada, March 20-23, 2026*, pages 499–509. IEEE.

Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, and 79 others. 2024. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*.

Seungjae Lee, Yoonkyo Jung, Inkook Chun, Yao-Chih Lee, Zikui Cai, Hongjia Huang, Aayush Talreja, Tan Dat Dao, Yongyuan Liang, Jia-Bin Huang, and Furong Huang. 2026. Tracegen: World modeling in 3d trace space enables learning from cross-embodiment videos. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 20721–20731. Computer Vision Foundation.

Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Yang Zhao, Sida Peng, Hengkai Guo, Xiaowei Zhou, Guang Shi, Jiashi Feng, and Bingyi Kang. 2026. Depth anything 3: Recovering the visual space from any views. In *The Fourteenth International Conference on Learning Representations, ICLR 2026, Rio de Janeiro, Brazil, April 23-27, 2026*. proceedings.iclr.cc.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. 2023. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. 2025. RDT-1B: a diffusion foundation model for bimanual manipulation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

Yicheng Liu, Zibin Dong, Baijun Ye, Tianyuan Yuan, Tao Jiang, Anqi Yang, Shicheng Cao, Haonan Liu, Yue Sun, Zihan Guo, Xiao Liu, Dong Ke, Changxun Pan, Chenru Wu, Tailai Cheng, Xiaoshu Ren, Xinlei Zhang, Jianning Cui, Zijie Zhao, and 8 others. 2026. G0.5: One autoregressive stream for robot reasoning and action. *Preprint*, arXiv:2608.11739.

Jiahao Lu, Jiayi Xu, Wenbo Hu, Ruijie Zhu, Chengfeng Zhao, Sai-Kit Yeung, Ying Shan, and Yuan Liu. 2026. Track4world: Feedforward world-centric dense 3d tracking of all pixels. In *Computer Vision - ECCV 2026 - 19th European Conference, Malmö, Sweden, September 8-12, 2026, Proceedings, Part LXIX*, volume 17069 of *Lecture Notes in Computer Science*, pages 184–204. Springer.

Yihang Luo, Shangchen Zhou, Yushi Lan, Xingang Pan, and Chen Change Loy. 2026. 4RC: 4D reconstruction via conditional querying anytime and anywhere. *arXiv preprint arXiv:2602.10094*.

Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. 2023. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *Conference on Robot Learning, CoRL 2023, 6-9 November 2023, Atlanta, GA, USA*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864. PMLR.

Arsalan Mousavian, Clemens Eppner, and Dieter Fox. 2019. 6-dof graspnet: Variational grasp generation for object manipulation. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 2901–2910. IEEE.

Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, and 275 others. 2023. Open X-Embodiment: Robotic learning datasets and RT-X models. https://arxiv.org/abs/2310.08864.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, and 7 others. 2024. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*.

Yuhao Pan, Haosong Peng, Zhengshen Zhang, Zhengyang Yan, Yalun Dai, Fushuo Huo, Chujie Wang, Tianyu Qi, Xiucheng Wang, Nan Cheng, and 1 others. 2026. World-to-wrist: Task-conditioned future wrist modeling for fine-grained robot manipulation. *arXiv preprint arXiv:2608.05369*.

Haosong Peng, Hao Li, Yalun Dai, Yushi Lan, Yihang Luo, Tianyu Qi, Zhengshen Zhang, Yufeng Zhan, Junfei Zhang, Wenchao Xu, and Ziwei Liu. 2026. Omnivggt: Omni-modality driven visual geometry grounded transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 36485–36497. Computer Vision Foundation.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron C. Courville. 2018. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, pages 3942–3951. AAAI Press.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67.

Edgar Sucar, Eldar Insafutdinov, Zihang Lai, and Andrea Vedaldi. 2026. V-DPM: 4d video reconstruction with dynamic point maps. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 14502–14511. Computer Vision Foundation.

Balakumar Sundaralingam, Adithyavairavan Murali, and Stan Birchfield. 2026. cuRoboV2: Dynamics-aware motion generation with depth-fused distance fields for High-DoF robots. *Preprint*, arXiv:2603.05493.

Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes. In *IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021*, pages 13438–13444. IEEE.

Bin Tan, Changjiang Sun, Xiage Qin, Hanat Adai, Zelin Fu, Tianxiang Zhou, Han Zhang, Yinghao Xu, Xing Zhu, Yujun Shen, and Nan Xue. 2026. Masked depth modeling for spatial perception. *arXiv preprint arXiv:2601.17895*.

Chenyi Wang, Xinkai Wang, Bokai Lin, Jialin Tian, Fucheng Zhang, Cewu Lu, and Lixin Yang. 2026a. Track4Action: Distilling world-centric 3d tracker into vision-language-action policies. *arXiv preprint arXiv:2608.03727*.

Jianyuan Wang, Minghao Chen, Shangzhan Zhang, Nikita Karaev, Johannes Schönberger, Patrick Labatut, Piotr Bojanowski, David Novotny, Andrea Vedaldi, and Christian Rupprecht. 2026b. Vggt-Ω. *arXiv preprint arXiv:2605.15195*.

Jingbo Wang, Wenxuan Song, Wenhao Yu, Han Zhao, Xi Wang, Jiayi Chen, Donglin Wang, Yan Wang, and Haoang Li. 2026c. Discrete forcing: Infusing discrete guidance into continuous denoising for few-step action experts. *Preprint*, arXiv:2609.39526.

Ruixiang Wang, Yongyi Su, Wenlve Zhou, Bo Yue, Hengyan Liu, Dekun Lu, Yuxin Tian, Yihan Fang, Zerui Wu, Xing Hu, and 1 others. 2026d. UNITAS: A 3D-Native world action model for embodied manipulation. *arXiv preprint arXiv:2610.12099*.

Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. 2024. Any-point trajectory modeling for policy learning. In *Robotics: Science and Systems XX, Delft, The Netherlands, July 15-19, 2024*.

Xiaomi Embodied Intelligence Team, University of Macau, Shaoqing Xu, Fang Li, Guozhi Zhan, Zhixiang Duan, Yuhan Wang, Yuechen Luo, Shengyin Jiang, Hanbing Li, Zhiying Du, Longlong Wang, Longmei Jiang, Weixiang Liang, Ying Gong, Yong Pan, Ziping Zhao, Zhiyuan Chen, Yangwei You, and 4 others. 2026. One policy, many embodiments: Unified camera-centric action geometry pre-training for heterogeneous embodied manipulation. *Preprint*, arXiv:2608.26058.

Sicheng Xie, Lingchen Meng, Zijie Diao, Haidong Cao, Zhiying Du, Shuyuan Tu, Jiaqi Leng, Qiuyue Wang, Mingsheng Li, Shuai Bai, Zuxuan Wu, and Yu-Gang Jiang. 2026. Unify robot actions in camera frame. *Preprint*, arXiv:2511.17001.

Ganlin Yang, Zhangzheng Tu, Yuqiang Yang, Sitong Mao, Junyi Dong, Tianxing Chen, Jiaqi Peng, Jing Xiong, Jiafei Cao, Jifeng Dai, Wengang Zhou, Yao Mu, and Tai Wang. 2026a. EventVLA: Event-driven visual evidence memory for long-horizon vision-language-action policies. *Preprint*, arXiv:2606.20092.

Jenai Xuning Yang, Rishit Dagli, Alex Zook, Hugo Hadfield, Ankit Goyal, Stan Birchfield, Fabio Ramos, and Jonathan Tremblay. 2026b. Robolab: A high-fidelity simulation benchmark for analysis of task generalist policies. *arXiv preprint arXiv:2604.09860*.

Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, and 17 others. 2026. World action models are zero-shot policies. *Preprint*, arXiv:2602.15922.

Tianhe Yu, Ted Xiao, Jonathan Tompson, Austin Stone, Su Wang, Anthony Brohan, Jaspiar Singh, Clayton Tan, Dee M, Jodilyn Peralta, Karol Hausman, Brian Ichter, and Fei Xia. 2023. Scaling robot learning with semantically imagined experience. In *Robotics: Science and Systems XIX, Daegu, Republic of Korea, July 10-14, 2023*.

Shanglin Yuan, Weiheng Zhao, Xianda Guo, Wei Sui, Li Yu, Wenyu Liu, and Xinggang Wang. 2026a. MotionVLA: Injecting geometric motion into vision-language-action model. *Preprint*, arXiv:2606.08288.

Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. 2026b. Fast-WAM: Do world action models need test-time future imagination? *Preprint*, arXiv:2603.16666.

Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 2024. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Robotics: Science and Systems XX*. Robotics: Science and Systems Foundation.

Chuhan Zhang, Guillaume Le Moing, Skanda Koppula, Ignacio Rocco, Liliane Momeni, Junyu Xie, Shuyang Sun, Rahul Sukthankar, Joëlle K. Barral, Raia Hadsell, Zoubin Ghahramani, Andrew Zisserman, Junlin Zhang, and Mehdi S. M. Sajjadi. 2026a. Efficiently reconstructing dynamic scenes one D4RT at a time. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2026, Denver, CO, USA, June 3-7, 2026*, pages 7382–7392. Computer Vision Foundation.

Chuye Zhang, Xiaoxiong Zhang, Wei Pan, Linfang Zheng, and Wei Zhang. 2025. Generative visual foresight meets task-agnostic pose estimation in robotic table-top manipulation. *Preprint*, arXiv:2509.00361.

Jianing Zhang, Chenhao Zheng, Yajun Yang, Max Argus, Rustin Soraki, Winson Han, Taira Anderson, Chun-Liang Li, Shuo Liu, Jiafei Duan, Zhongzheng Ren, Jieyu Zhang, and Ranjay Krishna. 2026b. MolmoMotion: Forecasting point trajectories in 3D with language instruction. *Preprint*, arXiv:2606.18558.

Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems XIX, Daegu, Republic of Korea, July 10-14, 2023*.

Hongyan Zhi, Peihao Chen, Siyuan Zhou, Yubo Dong, Quanxi Wu, Lei Han, and Mingkui Tan. 2025. 3dflowaction: Learning cross-embodiment manipulation from 3d flow world model. *arXiv preprint arXiv:2506.06199*.

Shangchen Zhou, Chongyi Li, Kelvin C. K. Chan, and Chen Change Loy. 2023. Propainter: Improving propagation and transformer for video inpainting. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*, pages 10443–10452. IEEE.

Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. 2019. On the continuity of rotation representations in neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 5745–5753. Computer Vision Foundation / IEEE.

Appendix

Contents

A Detailed Training Objectives **17**
- A.1 Geometry Supervision . . . 17
- A.2 TCP Tracking Supervision . . . 18
- A.3 Conditional Flow-matching Supervision . . . 19

B Cross-Embodiment Augmentation **19**
- B.1 Augmentation Pipeline . . . 19
- B.2 Motion Recovery from RGB . . . 20

C Implementation Details **20**
- C.1 Model Architecture and Hyperparameters . . . 20
- C.2 Frame Sampling Strategy . . . 21
- C.3 Sliding Window Inference . . . 22

D Tracking Evaluation **23**
- D.1 Datasets . . . 23
- D.2 Evaluation Protocol . . . 23
- D.3 Tracking Success Rate (SR) . . . 24
- D.4 Depth Metrics . . . 24
- D.5 3D TCP Tracking Metrics . . . 25
- D.6 $\mathrm{SE}(3)$ Trajectory Metrics . . . 25
- D.7 Gripper-state Accuracy . . . 26
- D.8 TCP4D Tracker Evaluation Results . . . 26
- D.9 Few-Shot Cross-Embodiment Adaptation . . . 27

E Cross-Embodiment Data Training Details **30**
- E.1 Augmented Dataset . . . 30
- E.2 Downstream Policy Training . . . 30

F Manipulation Evaluation **31**
- F.1 TCP4D Generator Result on RoboTwin . . . 31
- F.2 Real-world Task Performance . . . 31

G Limitations and Future Work **34**

A Detailed Training Objectives

This section details the losses in Eqs. 6 and 7, with an emphasis on TCP tracking supervision. We use the same TCP state $\mathbf{x}_i^a = (\mathbf{T}_i^a, g_i^a)$ and pose $\mathbf{T}_i^a = (\mathbf{R}_i^a, \mathbf{t}_i^a)$ as in the main paper. Here, $\mathbf{R}_i^a \in \mathrm{SO}(3)$ and $\mathbf{t}_i^a \in \mathbb{R}^3$ are the TCP orientation and position in frame i's camera coordinate system, and $g_i^a \in [0, 1]$ is gripper state. The index a identifies an available arm, with $a \in \{l, r\}$ for bimanual manipulation. For supervised outputs, hats indicate predictions and unhatted quantities denote ground truth.

A.1 Geometry Supervision

Given an RGB video $\mathbf{I} = \{I_i\}_{i=1}^N$, the geometry head predicts depth maps $\hat{\mathbf{D}} = \{\hat{\mathbf{D}}_i\}_{i=1}^N$ and camera-ray maps $\hat{\mathbf{M}} = \{\hat{\mathbf{M}}_i\}_{i=1}^N$, where $\hat{\mathbf{D}}_i \in \mathbb{R}^{H \times W}$ and $\hat{\mathbf{M}}_i \in \mathbb{R}^{\frac{H}{2} \times \frac{W}{2} \times 6}$, and H, W are image height and width. Each ray contains a viewing direction and a camera origin expressed in the first camera frame. We use $\mathbf{M}_i$ for ray maps and reserve $\mathbf{R}_i^a$ for TCP rotations. We denote depth and ray confidence maps by $\hat{\mathbf{Y}}_d$ and $\hat{\mathbf{Y}}_r$, respectively.

Following 4RC (Luo et al., 2026) and Depth Anything 3 (Lin et al., 2026), the geometry objective combines depth regression, spatial depth gradients, and ray regression:

$$\mathcal{L}_{\text{geo}} = \mathcal{L}_{\text{depth}} + \mathcal{L}_{\text{grad}} + \mathcal{L}_{\text{ray}}. \tag{8}$$

All three terms have unit weight. Depth and camera translations retain metric scale.

Confidence-weighted Regression. For a prediction $\hat{x} \in \mathbb{R}^d$, target x, and positive confidence $\hat{y}$, define

$$\begin{aligned} \ell_{\text{reg}}(\hat{x}, x) &= \frac{1}{d}\|\hat{x} - x\|_1, \\ \ell_{\text{conf}}(\hat{x}, x, \hat{y}) &= \hat{y}\, \ell_{\text{reg}}(\hat{x}, x) - \alpha \log \hat{y}, \end{aligned} \tag{9}$$

where $\alpha = 0.2$, matching the confidence regularization used for TCP supervision below. We use $d = 1$ for depth and $d = 6$ for rays. Let p index a pixel together with its video frame, and let Ω_d and Ω_r denote the valid depth and ray-grid locations, respectively.

Depth Loss. For predicted depths $\hat{\mathbf{D}}$, ground truth $\mathbf{D}$, and confidence $\hat{\mathbf{Y}}_d$, we average over valid depth pixels:

$$\mathcal{L}_{\text{depth}} = \frac{1}{|\Omega_d|} \sum_{p \in \Omega_d} \ell_{\text{conf}}(\hat{D}_p, D_p, \hat{Y}_{d,p}). \tag{10}$$

Depth-gradient Loss. To preserve sharp edges and smooth planar regions, we penalize the discrepancy between predicted and ground-truth depth gradients:

$$\mathcal{L}_{\text{grad}} = \|\nabla_x \hat{\mathbf{D}} - \nabla_x \mathbf{D}\|_1 + \|\nabla_y \hat{\mathbf{D}} - \nabla_y \mathbf{D}\|_1. \tag{11}$$

Here, ∇_x and ∇_y are horizontal and vertical finite-difference operators. In practice, we evaluate this term over $J = 4$ dyadic scales (stride 2^j, $j = 0, \ldots, J-1$) and average the results, using only differences whose endpoints have valid depth.

Ray Loss. For the six-channel ray predictions $\hat{\mathbf{M}}$, ground truth $\mathbf{M}$, and confidence $\hat{\mathbf{Y}}_r$, we use

$$\mathcal{L}_{\text{ray}} = \frac{1}{|\Omega_r|} \sum_{p \in \Omega_r} \ell_{\text{conf}}(\hat{\mathbf{M}}_p, \mathbf{M}_p, \hat{Y}_{r,p}). \tag{12}$$

The factor $1/d$ in Eq. 9 averages the residual over the six ray channels.

A.2 TCP Tracking Supervision

TCP poses are supervised in each observed frame's own camera coordinate system. The arm index a ranges over the available arms (e.g., left arm and right arm), so the same formulation applies to single-arm and bimanual tracking.

Virtual-point Pose Supervision. Following point-based pose supervision (Mousavian et al., 2019; Sundermeyer et al., 2021), we supervise TCP translation and orientation by aligning rigidly transformed reference points. As shown in Fig. 9, a TCP center alone provides no orientation constraint. We therefore expand each TCP into a rigid set of seven virtual points: its center and six points displaced along the positive and negative local coordinate axes. Their offsets are

$$\{\boldsymbol{\delta}_j\}_{j=1}^{7} = \{\mathbf{0},\ \pm r\mathbf{e}_k :\ k = 1, 2, 3\}, \tag{13}$$

where $\mathbf{e}_k$ are Cartesian basis vectors and $r = 0.03\,\mathrm{m}$.

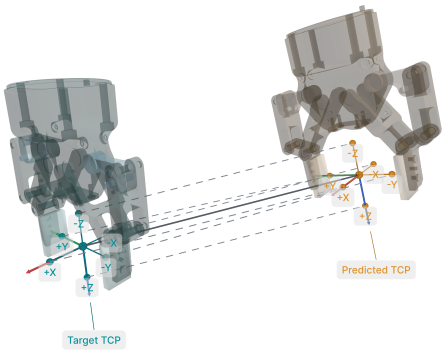

Figure 9: **TCP tracking supervision illustration.** The green Robotiq gripper on the left indicates the ground-truth TCP, while the orange Robotiq gripper on the right indicates the predicted TCP. We expand each TCP into a rigid set of seven virtual points, thereby augmenting TCP pose $\mathrm{SE}(3)$ supervision without additional annotations.

The target and predicted points are obtained by applying the corresponding TCP poses:

$$\begin{aligned} \mathbf{y}_{i,j}^{a} &= \mathbf{t}_i^a + \mathbf{R}_i^a \boldsymbol{\delta}_j, \\ \hat{\mathbf{y}}_{i,j}^{a} &= \hat{\mathbf{t}}_i^a + \hat{\mathbf{R}}_i^a \boldsymbol{\delta}_j. \end{aligned} \tag{14}$$

These targets require no additional annotations. Their spatial extent makes point alignment sensitive to both TCP translation and orientation. We measure the normalized point error by

$$e_{i,a}^{\text{point}} = \frac{1}{7}\sum_{j=1}^{7} \bar{\rho}\left(\frac{\hat{\mathbf{y}}_{i,j}^{a} - \mathbf{y}_{i,j}^{a}}{s_p}\right), \tag{15}$$

where $s_p = 0.1\,\mathrm{m}$ and $\bar{\rho}$ averages the unit-transition Smooth L1 penalty over the three coordinates. For a scalar residual z, this penalty is $z^2/2$ when $|z| < 1$ and $|z| - 1/2$ otherwise. We additionally supervise rotational alignment using the geodesic angle

$$d_R(\hat{\mathbf{R}}, \mathbf{R}) = \arccos\left(\frac{\mathrm{tr}(\hat{\mathbf{R}}^\top \mathbf{R}) - 1}{2}\right). \tag{16}$$

With predicted pose confidence $c_i^a > 0$, the pose objective is

$$\begin{aligned} e_{i,a}^{\text{pose}} &= e_{i,a}^{\text{point}} + \lambda_R \frac{d_R(\hat{\mathbf{R}}_i^a, \mathbf{R}_i^a)}{\pi}, \\ \mathcal{L}_{\text{pose}} &= \left\langle c_i^a e_{i,a}^{\text{pose}} - \alpha \log c_i^a \right\rangle_{\text{valid}}, \end{aligned} \tag{17}$$

where $\lambda_R = 0.5$ and $\alpha = 0.2$. The average includes valid frame–arm pairs across the minibatch; an invalid reference query excludes that arm's trajectory. The confidence regularizer discourages reducing confidence solely to suppress reconstruction errors.

Temporal Velocity Consistency. We also supervise motion through finite differences of the virtual points. Let s_i be the timestamp of sampled frame i and $\Delta_i = s_i - s_{i-1}$. The target and predicted velocities are

$$\begin{aligned} \mathbf{v}_{i,j}^{a} &= \frac{\mathbf{y}_{i,j}^{a} - \mathbf{y}_{i-1,j}^{a}}{\Delta_i}, \\ \hat{\mathbf{v}}_{i,j}^{a} &= \frac{\hat{\mathbf{y}}_{i,j}^{a} - \hat{\mathbf{y}}_{i-1,j}^{a}}{\Delta_i}. \end{aligned} \tag{18}$$

Using the actual, signed interval accommodates variable frame sampling and reversed clips. We define

$$
\begin{aligned}
e^{v}_{i,a} &= \frac{1}{7}\sum_{j=1}^{7} \bar{\rho}\left(\frac{\hat{\mathbf{v}}^{a}_{i,j} - \mathbf{v}^{a}_{i,j}}{s_v}\right), \\
\mathcal{L}_{\text{temp}} &= \left\langle \tilde{c}^{a}_{i} e^{v}_{i,a} - \alpha \log \tilde{c}^{a}_{i} \right\rangle_{\text{pair}},
\end{aligned}
\tag{19}
$$

where $s_v = 1\,\text{m/s}$ and $\tilde{c}^{a}_{i} = \min(c^{a}_{i}, c^{a}_{i-1})$. The average includes valid adjacent frame pairs for each available arm. Applying this loss to the extended points constrains temporal changes in both translation and orientation. These finite differences are computed in the supplied per-frame camera-coordinate labels.

Gripper and Stage-wise TCP Objectives. We supervise predicted gripper state $\hat{g}^{a}_{i}$ against $g^{a}_{i} \in [0,1]$ with binary cross-entropy, averaged over the same valid frame–arm pairs. Continuous targets use the unweighted loss; binary targets use positive-class reweighting by the ratio of negative to positive labels when both classes are present. Denoting this loss by $\mathcal{L}_{\text{grip}}$, the two TCP objectives are

$$
\begin{aligned}
\mathcal{L}^{(1)}_{\text{tcp}} &= \mathcal{L}_{\text{pose}} + 0.2\mathcal{L}_{\text{temp}} + 0.2\mathcal{L}_{\text{grip}}, \\
\mathcal{L}^{(2)}_{\text{tcp}} &= \mathcal{L}_{\text{pose}} + 0.2\mathcal{L}_{\text{grip}}.
\end{aligned}
\tag{20}
$$

Thus Stage 1 supervises pose, temporal motion, and gripper reconstruction, while Stage 2 retains pose and gripper reconstruction.

A.3 Conditional Flow-matching Supervision

Action Representation and Normalization. Ground-truth future TCP poses are transformed into the current observation's camera coordinate system before normalization. Each arm contributes a 10D action $(\mathbf{t}, \mathbf{r}, g)$: metric position $\mathbf{t} \in \mathbb{R}^3$, the first two columns of the rotation matrix $\mathbf{r} \in \mathbb{R}^6$, and gripper state $g \in [0,1]$. Thus the bimanual target trajectory $\mathbf{A}$ has shape $h \times 20$, where $h = 32$ is the prediction horizon.

Position is normalized separately for each arm and coordinate using the 1st and 99th percentiles of valid training targets, pooled across future steps and excluding padding. An affine transform maps this range approximately to $[-1,1]$, without clipping outliers. The 6D rotation remains unscaled, while gripper states are mapped linearly from $[0,1]$ to $[-1,1]$. At inference, the same statistics recover metric positions; rotation columns are orthonormalized and gripper states are thresholded at zero.

Flow-matching Objective. The condition $\mathcal{O}_t$ contains L past and current RGB frames, the current joint and gripper state $\mathbf{s}_t$, and wrist-camera images $I^{w,a}_{t}$; ℓ is the language instruction. Gaussian noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and flow time $\tau \in [0,1)$ define the interpolated input $\mathbf{A}_\tau$ and target $\mathbf{v}^\star$ in Eq. 5. We write $\hat{\mathbf{v}} = \mathbf{v}_\phi(\mathbf{A}_\tau, \tau; \mathcal{O}_t, \ell)$. The generation objective is

$$
\mathcal{L}_{\text{fm}} = \mathbb{E}_{\mathbf{A},\boldsymbol{\epsilon},\tau}\left[\sum_{q\in\{\text{pos,rot,grip}\}} \frac{\left\langle \|\hat{\mathbf{v}}_q - \mathbf{v}^{\star}_{q}\|_2^2 \right\rangle_f}{d_q}\right],
\tag{21}
$$

where $d_{\text{pos}} = 3$, $d_{\text{rot}} = 6$, and $d_{\text{grip}} = 1$ are the per-arm component dimensions. For each trajectory, $\langle\cdot\rangle_f$ averages over valid future steps and available arms; padded steps are excluded. The expectation also averages over trajectories and sampled noise and flow time. All three component means have unit weight; dividing by d_q prevents the 6D rotation from receiving a larger weight solely because of its dimensionality.

B Cross-Embodiment Augmentation

B.1 Augmentation Pipeline

TCP4D enables cross-embodiment augmentation by recovering task-space motion directly from RGB demonstrations. As illustrated in Fig. 3, the pipeline comprises motion recovery, embodiment retargeting, and visual composition. Given a source video and initial TCP queries, TCP4D predicts TCP trajectories, gripper states, and camera geometry. The recovered poses are aligned in a common coordinate frame

and transferred to the target robot through inverse kinematics using its kinematic model, producing joint configurations that follow the demonstrated task-space motion.

We segment the source robot using SAM3 (Carion et al., 2026) or, when available, its geometric model and recorded states. ProPainter (Zhou et al., 2023) completes the masked regions to reconstruct the background. We then render the target robot at the recovered configurations and composite it into the completed frames using the aligned camera geometry. The resulting images are paired with target configurations to form augmented demonstrations. Depth from TCP4D can optionally improve segmentation quality or distinguish foreground and background, but the pipeline does not depend on depth.

B.2 Motion Recovery from RGB

The key advantage of TCP4D is that motion information for retargeting can be inferred from video rather than supplied as recorded source-robot states. Table 4 compares the required resources and the sources of motion and camera information. Predicted TCP trajectories and gripper states provide the motion representation, while inverse kinematics recovers target joint configurations. Camera geometry can also be predicted when calibration is unavailable. With SAM3 segmentation, source URDFs and recorded joint configurations are unnecessary for robot removal. This extends augmentation to RGB demonstrations without accompanying robot-state trajectories, while retaining the target model for retargeting and rendering.

Table 4: **Resources and motion signals for cross-embodiment augmentation.** ✓: supplied as input or recorded metadata (required); ×: not required; **Pred.**: inferred from RGB by TCP4D. Source URDFs are unnecessary with image-based segmentation.

Resource / signal	**RoVi-Aug** (Chen et al., 2024b)	**OXE-AugE** (Ji et al., 2025)	TCP4D
Source robot URDF	×	×	×
Target robot URDF	×	✓	✓
Recorded camera intrinsics	×	×	**Pred.**
Recorded camera extrinsics	×	×	**Pred.**
Source TCP trajectory	✓	✓	**Pred.**
Source joint configurations	✓	×	×
Source gripper state	✓	✓	**Pred.**

C Implementation Details

We provide the architecture configuration, frame sampling strategy, preprocessing, and optimization details in this section, followed by the dataset splits and evaluation settings.

C.1 Model Architecture and Hyperparameters

Dataset-specific Training. We train two independent Stage 1 models: a single-arm TCP tracker on DROID and a bimanual TCP tracker on RoboTwin. Both models initialize their visual backbones and geometry modules from pretrained 4RC weights. Stage 2 uses only RoboTwin data and jointly fine-tunes the TCP tracker and scene geometry modules while training the conditioning modules and TCP generator. The camera head is excluded from optimization in both stages and is enabled for camera evaluation. The pretrained wrist-image and language encoders remain frozen during Stage 2. Table 5 summarizes the model architecture used for the 48-task RoboTwin experiments.

Model Size. We report total registered model parameters, including frozen modules and inherited heads retained in the implementation. The TCP4D Stage 1 trackers contain approximately 1.522B parameters. The full tracker–generator model contains approximately 2.272B parameters, comprising the Stage 1 tracker, 336.462M parameters in the added action and conditioning modules, a frozen DINOv2-Large wrist-image encoder with 304.369M parameters, and a frozen T5-base text encoder with 109.629M parameters. The DiT accounts for approximately 294M of the added parameters. The wrist-image encoder

Table 5: **Model architecture for the TCP4D Stage 1&2 experiments.**

Module	Architecture / setting	Specification
Shared visual and TCP modules	Visual backbone and geometry head	4RC backbone and Dual-DPT; trainable
	Sparse TCP modules	Query encoder, decoder, and TCP head; trainable
	TCP query neighborhood	3×3 patches per arm and frame
	Geometry decoding	Enabled
	Camera head	Excluded from optimization in both stages
Conditioning encoders	Robot-state encoder	MLP: $14 \to 1024 \to 1280$
	Global visual token grid	14×19 (266 tokens)
	Wrist-image encoder	Shared DINOv2-Large; frozen
	Language encoder	T5-base; frozen
	Maximum language length	128 tokens
	Image and language condition width	2048
Trajectory generator (DiT)	Transformer blocks	12; alternating cross- and self-attention
	Hidden width	1280
	Attention heads	32 (40 dimensions per head)
	Feed-forward hidden width	5120 (expansion ratio 4)
	Dropout	0.2
	Output feature width	1024
	Action decoder	MLP: $1024 \to 1024 \to 20$
Input and output sequences	Observation history / prediction horizon	$L = 4$ frames / $h = 32$ steps
	Local TCP history tokens	$4 \times 2 \times 9 = 72$
	DiT input sequence	1 state + 72 TCP + 32 action tokens
	Image context tokens	$266 + 2 \times 266 = 798$
	Predicted TCP trajectory	32×20
	Flow-matching inference	4 Euler steps

is shared across the left and right cameras and counted once. Table 6 provides the module-wise breakdown, including the inherited dense tracking head retained in the implementation. Per-module counts are rounded independently, so their displayed sum may differ slightly from the reported total.

Table 6: **Model parameter counts.** Counts include frozen modules and retained heads; the wrist encoder is shared across both cameras.

Module	Parameters (M)
Visual backbone	1,136.491
Motion decoder	283.340
TCP query encoder	0.808
TCP trajectory prediction head	0.800
Depth and ray geometry head	50.115
Camera parameter decoder	18.908
Dense tracking head (disabled)	31.212
Shared wrist DINOv2-Large	304.369
T5-base text encoder	109.629
Added action and conditioning modules	336.462
Total	**2,272.133**

Hyperparameters. Table 7 summarizes the shared and stage-specific training hyperparameters.

C.2 Frame Sampling Strategy

Stage 1 Sampling. Each minibatch contains a fixed total of 18 frames per device. For each minibatch, we randomly choose the number of clips $S \in \{1, 2, 3, 6, 9\}$ and sample $18/S$ frames per clip, yielding the respective clip lengths 18, 9, 6, 3, and 2. Each clip comes from a single camera stream, with a temporal stride sampled uniformly from the feasible values in $\{1, 2, 3, 4, 5\}$ and a randomly chosen starting frame. The sampled stride is constant within a clip and can differ across clips. Each clip is

Table 7: **Training hyperparameters.** Common settings apply to both DROID Stage 1 and the RoboTwin Stage 2 experiments.

Scope	Hyperparameter	Value
Common	Optimizer	AdamW
	Adam coefficients (β_1, β_2)	$(0.9, 0.95)$
	Adam ϵ	10^{-8}
	Weight decay	0.01
	Mixed precision	bfloat16
	Gradient accumulation steps	2
	Gradient clipping norm	1.0
	Learning-rate schedule	Cosine
	Warm-up duration	1,000 optimizer steps
	Minimum / peak learning rate	0.1
	Training duration	500,000 optimizer steps
DROID Stage 1	Peak LR: visual backbone	10^{-5}
	Peak LR: geometry head	2×10^{-5}
	Peak LR: sparse TCP modules	10^{-4}
RoboTwin Stage 2	Batch size per device	8 samples
	Observation history per sample	4 frames
	Peak LR: geometry head	2×10^{-5}
	Peak LR: shared TCP decoder	10^{-5}
	Peak LR: backbone and other trainable modules	10^{-4}
	Frozen modules	Camera head; pretrained wrist-image and language encoders

reversed with probability 0.5; images and all associated labels are reordered together. The first frame in the resulting sequence serves as the TCP query frame, so queries can originate at different points within an episode and support tracking in either temporal direction. Temporal supervision uses the signed time intervals between sampled frames, accounting for both subsampling and reversal. Clips remain within valid trajectory segments and do not cross detected discontinuities.

Stage 2 Sampling. For each sample, we randomly choose a current frame t and use the four consecutive observations ending at t as history ($L = 4$), with the following 32 TCP states as targets ($h = 32$). Sampling is strictly forward with unit temporal stride and no reversal. If fewer than four historical frames are available within the current valid segment, we repeat its earliest observed frame. If fewer than 32 future states remain, we repeat the last valid state for storage and mask the padded positions in attention and loss computation. The current robot state and wrist images are taken at t, and a language instruction is sampled from those associated with the episode.

C.3 Sliding Window Inference

As illustrated in Fig. 10, TCP4D extends TCP tracking to long videos by propagating visual queries across overlapping windows. Only the initial frame requires 2D TCP queries for the available arms; subsequent windows use queries derived from the model's recovered TCP positions.

TCP Tracking. For example, the tracker processes $n - 1$ frame windows by default, with a configurable length of 2–18 frames. Consecutive windows share exactly one boundary frame, so a full $n - 1$ frame window advances the sequence by $n - 2$ frames; the final window can be shorter. After processing a window, we project its final-frame 3D TCP positions into the shared boundary image using the camera intrinsics decoded from the ray map. These projected positions become the reference-frame queries for the next window, as shown in Fig. 10.

Conditional TCP Generation. For each window, we project the predicted TCP positions in the L history frames into their corresponding images and sample local feature tokens from the shared visual representations. These tokens are fed into the DiT to condition future TCP generation.

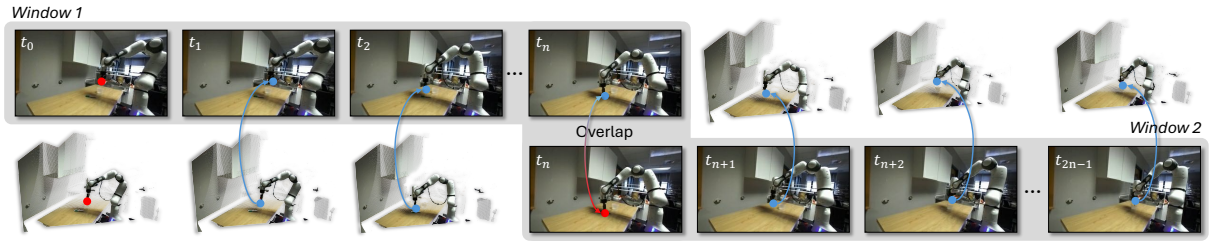

Figure 10: **Sliding-window TCP tracking inference.** TCP4D supports inference on sequences of varying lengths. To track a long sequence, only the TCP's 2D location in the first frame needs to be specified. The predicted 3D TCP position, (x, y, z), at the final frame can then be projected onto the camera plane and used as the new query for the next window.

Figure 11: **Query points on DROID and Robolab for baseline evaluation.** For baselines evaluated by rigid fitting of point tracks, we select 16 point groups on the gripper in the first frame of each window using ground-truth annotations, track them throughout the window, and fit the TCP pose from the tracked points.

D Tracking Evaluation

This section describes the datasets, protocol, and metrics used for the tracking evaluation in Table 1 and reports additional results. All TCP poses are expressed in the camera coordinate system of the corresponding frame.

D.1 Datasets

DROID. The PointWorld-annotated DROID subset contains 33,966 episodes. We use a fixed episode-level split with seed 42, assigning approximately 90% of the episodes to training. From the remaining 10%, we randomly sample 100 episodes for evaluation. Each DROID episode has two third-person view cameras and is sampled at 15 Hz.

RoboLab. The additional RoboLab set contains one episode for each of 100 distinct tasks and is used exclusively for in-the-wild evaluation. We use $\pi_{0.5}$ (Intelligence et al., 2025) as the policy to generate trajectories, including both successful and failed trajectories. During data collection, we randomize the backgrounds and camera positions. RoboLab is sampled at 15 Hz.

D.2 Evaluation Protocol

Baseline Evaluation Windows. The 4RC (Luo et al., 2026), Any4D (Karhade et al., 2026), Track4World (Lu et al., 2026), VDPM (Sucar et al., 2026), MoSE3 (Cheng et al., 2026), and CoTracker3 (Karaev et al., 2025) baselines use independent, non-overlapping nine-frame windows, with a potentially shorter final window. Each window's query frame is excluded from tracking metrics.

TCP Pose Recovery from Point Tracks. For 4RC, Any4D, Track4World, VDPM, and CoTracker3, we recover TCP poses by rigid fitting of point tracks. As shown in Fig. 11, each window is initialized with ground-truth-projected points on the fixed Robotiq gripper body. Each query is associated with a known surface point on the gripper URDF model, and this query-to-model correspondence is retained throughout the window. CoTracker3 uses VGGT-Ω depth predicted for every frame, together with the calibrated

camera intrinsics, to lift its 2D tracks into 3D. For all these baselines, including CoTracker3, we calibrate the predicted 3D point group's scale using the known metric dimensions of the corresponding URDF points at window initialization and keep this scale fixed throughout the window.

At each tracked frame, we robustly fit a rigid transformation from the corresponding gripper model points to their scale-calibrated 3D predictions. The fitted rotation and translation describe the gripper body's pose in that frame's camera coordinates. We apply the known fixed body-to-TCP offset to recover the TCP position and use the fitted body rotation as the TCP orientation, consistent with the dataset's TCP convention. Repeating this fit across frames converts the point-group tracks into a TCP pose trajectory.

MoSE3 Native $\mathrm{SE}(3)$ Motion Estimation. For MoSE3, we use its official rigidity-weighted per-point $\mathrm{SE}(3)$ estimation procedure. At the start of each window, we perform the same URDF-based scale calibration and rigid fitting to initialize the TCP pose from the predicted 3D point group. We then propagate this pose using the $\mathrm{SE}(3)$ motion associated with a fixed gripper-surface query, selected as the query nearest the image-space mean of the group, and transform it into each frame's camera coordinates using the predicted camera poses. The initialization scale is applied to both motion and camera translations. No additional CAD-to-track rigid fitting is performed at subsequent frames. The same evaluation windows, query-frame exclusions, and metric definitions apply.

Metrics and Alignment. We evaluate the recovered TCP positions with APD and MPE, and the recovered pose trajectory with ATE, RRE, and RTE in Table 1. APD uses one GT-based global-median scale per episode–camera trajectory, computed by Eq. 25. MPE is computed without evaluation-time alignment; each method retains the scale of its recovered or directly predicted TCP trajectory. ATE, RRE, and RTE share one position-based Sim(3) alignment per episode trajectory: its scale, rotation, and translation transform positions, and its rotation transforms orientations. 3D Tracking and $\mathrm{SE}(3)$ for those baselines are computed only over each method's successful windows. SR reports the percentage of successfully processed windows.

TCP4D uses nine-frame windows with one shared boundary frame and propagates predicted queries after the initial ground-truth query, following the *Sliding Window Inference* strategy in Appx. C.3. Only its initial query frame is excluded, and overlapping frames are scored once using the preceding window's prediction.

D.3 Tracking Success Rate (SR)

For each camera sequence, we compute

$$\mathrm{SR} = N_{\mathrm{succ}}/(N_{\mathrm{succ}} + N_{\mathrm{fail}}), \tag{22}$$

and report the unweighted mean across episode. Eligible targets have finite ground-truth TCP positions with positive depth and projections inside the image, with query-frame exclusions as described above; windows without eligible targets are excluded. A window succeeds if initialization is valid and every eligible target has a finite TCP prediction with positive depth. Failures include 1) invalid gripper-query initialization, 2) failed CAD-scale estimation, 3) failed initial TCP recovery or inference, and 4) missing, non-finite, or non-positive-depth target predictions. Failed windows remain in the SR denominator but are excluded from the metrics in all the tables.

D.4 Depth Metrics

We reuse Ω_d from Appx. A.1, restricted to each frame and ground-truth depths in $(0, 3]$ m. Per-frame median alignment uses

$$\tilde{D}_p = s_D \hat{D}_p, \qquad s_D = \frac{\mathrm{median}_{q\in\Omega_d} D_q}{\mathrm{median}_{q\in\Omega_d} \hat{D}_q}. \tag{23}$$

Only a multiplicative scale is fitted from GT depth. For metric-depth evaluation in Table 8, we instead use the native prediction $\hat{D}_p$ directly, with no GT-based scale correction. Define $r_p = \max(\tilde{D}_p/D_p, D_p/\tilde{D}_p)$

for $\tilde{D}_p > 0$, and $r_p = \infty$ otherwise. The three depth metrics are

$$\begin{aligned} \text{AbsRel} &= \frac{1}{|\Omega_d|} \sum_{p \in \Omega_d} \frac{|\tilde{D}_p - D_p|}{D_p}, \\ \text{MAE} &= \frac{10^3}{|\Omega_d|} \sum_{p \in \Omega_d} |\tilde{D}_p - D_p|, \\ \delta_1 &= \frac{100}{|\Omega_d|} \sum_{p \in \Omega_d} \mathbf{1}[r_p < 1.25]. \end{aligned} \tag{24}$$

MAE is reported in millimeters and δ_1 in percent.

D.5 3D TCP Tracking Metrics

We retain the TCP notation from Appx. A, omitting the arm index a below; positions are in meters. Let $\mathcal{E}$ contain the target frames from successful tracking windows, excluding query frames as described above.
APD. We average distance-threshold accuracies at $\mathcal{D} = \{5, 10, 20, 50\}$ mm, using one global-median scale per episode–camera trajectory:

$$s_{\text{GM}} = \frac{\text{median}_{i \in \mathcal{E}} \, \|\mathbf{t}_i\|_2}{\text{median}_{i \in \mathcal{E}} \, \|\hat{\mathbf{t}}_i\|_2}. \tag{25}$$

The scale is fitted across retained successful windows, with no rotation or translation alignment. Writing $e_i = 10^3 \|s_{\text{GM}} \hat{\mathbf{t}}_i - \mathbf{t}_i\|_2$ in millimeters, we compute

$$\text{APD} = \frac{100}{4|\mathcal{E}|} \sum_{d \in \mathcal{D}} \sum_{i \in \mathcal{E}} \mathbf{1}[e_i \le d]. \tag{26}$$

Mean Position Error (MPE). MPE is the mean Euclidean TCP position error in metric coordinates, reported in millimeters:

$$\text{MPE} = \frac{10^3}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} \|\hat{\mathbf{t}}_i - \mathbf{t}_i\|_2. \tag{27}$$

No evaluation-time alignment is applied; rigid-group baselines retain their initialization scale calibration.

For the GM-aligned comparison in Table 9, we additionally report

$$\text{MPE}_{\text{GM}} = \frac{10^3}{|\mathcal{E}|} \sum_{i \in \mathcal{E}} \|s_{\text{GM}} \hat{\mathbf{t}}_i - \mathbf{t}_i\|_2. \tag{28}$$

The scale s_{GM} is fitted once per episode trajectory using GT positions across all retained successful windows, as in Eq. 25. PCK_d thresholds the GM-aligned error, while Acc_d thresholds the metric error, both at d mm.

D.6 $\mathrm{SE}(3)$ Trajectory Metrics

ATE, RRE, and RTE are reported as RMSE. We fit one position-based Umeyama Sim(3) $(s_{\text{aln}}, \mathbf{R}_{\text{aln}}, \mathbf{t}_{\text{aln}})$ per episode–camera sequence on $\mathcal{E}$, shared by all three metrics. The aligned poses $\bar{\mathbf{T}}_i = (\bar{\mathbf{R}}_i, \bar{\mathbf{t}}_i)$ satisfy

$$\begin{aligned} \bar{\mathbf{t}}_i &= s_{\text{aln}} \mathbf{R}_{\text{aln}} \hat{\mathbf{t}}_i + \mathbf{t}_{\text{aln}}, \\ \bar{\mathbf{R}}_i &= \mathbf{R}_{\text{aln}} \hat{\mathbf{R}}_i. \end{aligned} \tag{29}$$

Relative errors use the full pose residual

$$\mathbf{E}_{ij} = (\mathbf{T}_i^{-1} \mathbf{T}_j)^{-1} (\bar{\mathbf{T}}_i^{-1} \bar{\mathbf{T}}_j), \tag{30}$$

with $j = i + 1$ in the original frame indices and valid poses at both retained endpoints. RTE uses the translation norm of $\mathbf{E}_{ij}$, and RRE its rotation angle, computed using d_R in Eq. 16.

D.7 Gripper-state Accuracy

We threshold $\hat{g}_i$ and g_i at 0.5, treating values ≥ 0.5 as open and smaller values as closed. Acc. is the average agreement between the resulting binary states, reported as a percentage.

D.8 TCP4D Tracker Evaluation Results

We extend Table 1 with complete depth metrics, additional 3D TCP tracking and $\mathrm{SE}(3)$ statistics, and an evaluation of the camera head.

Metric depth. Table 8 reports metric depth evaluation on DROID and RoboLab, using native depth predictions in meters without alignment to GT. Results use the GT depth mask within 3 m. We compare DA3NESTED-GIANT-LARGE, Any4D, MapAnything, and TCP4D, for which unaligned metric-depth results are available. TCP4D achieves the lowest AbsRel, MAE, and RMSE among these methods on both datasets, with MAE of 28.37 mm on DROID and 113.92 mm on RoboLab. Its δ_1 accuracies are 97.79% and 90.53%, respectively. These results evaluate absolute scale accuracy together with depth reconstruction error.

Table 8: **Metric depth evaluation on DROID and RoboLab.** Native depth predictions are evaluated in meters without GT-based scale alignment, using the GT depth mask within 3 m. Best and second-best values are highlighted independently for each dataset.

Method	AbsRel ↓	MAE (mm) ↓	RMSE (mm) ↓	SqRel (m)↓	$\mathrm{RMSE}_{\log}$ ↓	$\log_{10}$ ↓	SILog ↓	$\delta_1(\%)$ ↑	$\delta_2(\%)$ ↑	$\delta_3(\%)$ ↑
DROID (Eval set)										
DA3-GIANT (Lin et al., 2026)	0.2925	280.36	408.87	0.1486	0.3773	0.1270	32.67	49.98	78.42	91.50
MapAnything (Keetha et al., 2026)	0.7756	616.20	732.90	0.7072	0.5838	0.2276	27.03	13.73	43.24	74.69
Any4D (Karhade et al., 2026)	1.1698	906.84	1039.32	1.4912	0.7530	0.3029	28.42	8.18	27.84	51.74
TCP4D	0.0303	28.37	110.75	0.0156	0.0995	0.0116	9.91	97.79	98.87	99.32
RoboLab (Unseen)										
DA3-GIANT (Lin et al., 2026)	0.3066	185.83	245.87	0.1868	0.3237	0.1134	24.26	54.08	83.15	94.75
MapAnything (Keetha et al., 2026)	1.1662	741.13	818.13	1.1475	0.7575	0.3172	19.52	1.57	10.28	43.82
Any4D (Karhade et al., 2026)	1.5807	985.87	1058.42	1.9600	0.9258	0.3926	19.07	0.51	3.33	15.09
TCP4D	0.2074	113.92	218.97	0.1752	0.2408	0.0659	21.44	90.53	95.24	96.16

Table 9: **Additional 3D TCP tracking metrics.** Results are averages over successful windows. PCK_d and Acc_d report percentages of position errors within d mm under GT scale alignment and metric scale, respectively. Best and second-best values are highlighted independently for each dataset.

	GT scale alignment (GM)					Metric scale (no GT alignment)				
Method	$\mathrm{MPE}_{\mathrm{GM}}(mm)$ ↓	$\mathrm{PCK}_5(\%)$ ↑	$\mathrm{PCK}_{10}(\%)$ ↑	$\mathrm{PCK}_{20}(\%)$ ↑	$\mathrm{PCK}_{50}(\%)$ ↑	Median(mm) ↓	P95(mm) ↓	$\mathrm{Acc}_{10}(\%)$ ↑	$\mathrm{Acc}_{20}(\%)$ ↑	$\mathrm{Acc}_{50}(\%)$ ↑
DROID (Eval set)										
4RC (Luo et al., 2026)	103.14	0.15	0.76	4.34	29.73	84.70	276.93	0.96	6.19	32.33
Track4World (Lu et al., 2026)	141.96	0.00	0.08	1.43	13.20	174.06	338.74	0.03	0.54	5.01
Any4D (Karhade et al., 2026)	171.01	0.01	0.05	0.52	8.69	229.73	430.68	0.02	0.05	1.71
VDPM (Sucar et al., 2026)	119.12	0.01	0.28	1.68	16.71	182.45	336.68	0.02	0.42	4.10
CoTracker3 (Karaev et al., 2025)	87.26	0.41	2.34	10.68	42.73	67.66	241.52	4.18	14.81	44.76
MoSE3 (Cheng et al., 2026)	86.58	0.17	1.18	7.19	40.98	83.69	245.35	1.12	5.95	32.29
TCP4D	10.76	38.28	74.55	89.78	96.52	10.71	22.34	75.16	89.52	94.99
RoboLab (Unseen)										
4RC (Luo et al., 2026)	127.50	0.02	0.22	1.80	17.63	137.70	370.72	0.52	3.11	19.50
Track4World (Lu et al., 2026)	141.79	0.00	0.04	0.58	9.16	192.11	406.60	0.02	0.18	3.08
Any4D (Karhade et al., 2026)	179.75	0.00	0.02	0.16	4.42	215.42	469.83	0.01	0.15	1.50
VDPM (Sucar et al., 2026)	84.42	0.08	0.55	3.84	35.15	99.02	235.26	0.32	2.46	22.80
CoTracker3 (Karaev et al., 2025)	114.20	0.15	0.78	4.43	29.77	101.00	345.16	1.72	6.95	30.20
MoSE3 (Cheng et al., 2026)	110.98	0.10	0.64	4.50	29.08	100.45	329.27	0.95	5.58	26.34
TCP4D	21.55	2.83	16.08	60.86	95.07	26.32	51.54	10.04	48.42	87.21

3D TCP tracking. Table 9 compares 3D TCP tracking in GT scale alignment and metric scale. GT scale alignment rescales each episode–camera trajectory by $s_{\mathrm{GM}} = \mathrm{median}\,\|\mathbf{t}_i\|_2 / \mathrm{median}\,\|\hat{\mathbf{t}}_i\|_2$. Metric scale uses TCP4D predictions in meters and baseline predictions scaled to the known CAD dimensions at each window's initialization, following the protocol in Sec. D.2. $\mathrm{MPE}_{\mathrm{GM}}$ and PCK_d use the rescaled positions; Median, P95, and Acc_d use metric positions.

$\mathrm{SE}(3)$ **trajectories.** Table 10 reports the mean, median, and P95 of RRE and RTE after per-camera Sim(3) alignment. Relative-pose errors are computed between adjacent frames within each inference window.

Table 10: **Additional** SE(3) **trajectory error statistics.** Mean, median, and P95 are computed per camera over successful windows and then macro-averaged.

Method	RRE (°) ↓			RTE (mm) ↓		
	Mean	Median	P95	Mean	Median	P95
DROID (Eval set)						
4RC (Luo et al., 2026)	4.74	2.15	18.37	9.24	6.79	26.35
Track4World (Lu et al., 2026)	6.40	3.50	21.93	8.65	5.98	25.04
Any4D (Karhade et al., 2026)	6.13	2.66	23.22	9.26	6.60	26.41
VDPM (Sucar et al., 2026)	4.45	1.85	17.71	8.03	5.70	23.08
CoTracker3 (Karaev et al., 2025)	11.67	8.51	32.37	16.30	11.38	45.61
MoSE3 (Cheng et al., 2026)	0.95	0.73	2.53	4.50	3.55	11.60
TCP4D	0.74	0.56	1.90	2.59	2.01	6.50
RoboLab (Unseen)						
4RC (Luo et al., 2026)	3.76	1.58	15.28	5.79	3.98	17.06
Track4World (Lu et al., 2026)	5.41	2.56	19.75	6.08	4.10	17.92
Any4D (Karhade et al., 2026)	4.02	0.94	18.38	6.10	4.56	16.97
VDPM (Sucar et al., 2026)	2.72	0.83	12.61	5.56	3.58	17.17
CoTracker3 (Karaev et al., 2025)	11.39	8.38	31.78	12.93	9.34	36.86
MoSE3 (Cheng et al., 2026)	0.71	0.50	2.01	3.58	2.78	9.41
TCP4D	1.83	0.72	4.79	3.44	2.38	10.21

Camera geometry. For this evaluation, we enable the retained camera head to predict camera extrinsics relative to the robot base and camera intrinsics. Table 11 reports mean errors on DROID and RoboLab using camera-wise macro averaging.

More TCP tracking visualizations. We show additional TCP4D tracking qualitative comparison in Fig. 14.

Table 11: **Camera pose evaluation for TCP4D.**

Metric	DROID	RoboLab
Extrinsics		
Translation (mm)	12.96	129.80
Rotation (°)	1.26	2.42
Camera center (mm)	14.80	130.99
Intrinsics		
f_x rel. error (%)	0.248	0.477
f_y rel. error (%)	0.179	0.412
Horizontal FoV (°)	0.139	0.268
Vertical FoV (°)	0.096	0.221

D.9 Few-Shot Cross-Embodiment Adaptation

After large-scale pretraining on the DROID dataset, TCP4D demonstrates strong few-shot cross-embodiment adaptation capabilities. We evaluate it on the Berkeley AUTOLAB UR5 dataset (Chen et al.), which contains 1,000 demonstration episodes across different tasks, comprising 97,939 video frames in total. We fine-tune TCP4D under two settings: (1) using random 3 episodes for three-shot transfer and (2) using random 10 episodes for few-shot transfer. The raw depth data captured by the sensor are post-processed using LingBot-Depth (Tan et al., 2026). Fig. 12 visualizes the three-shot and few-shot transfer results.

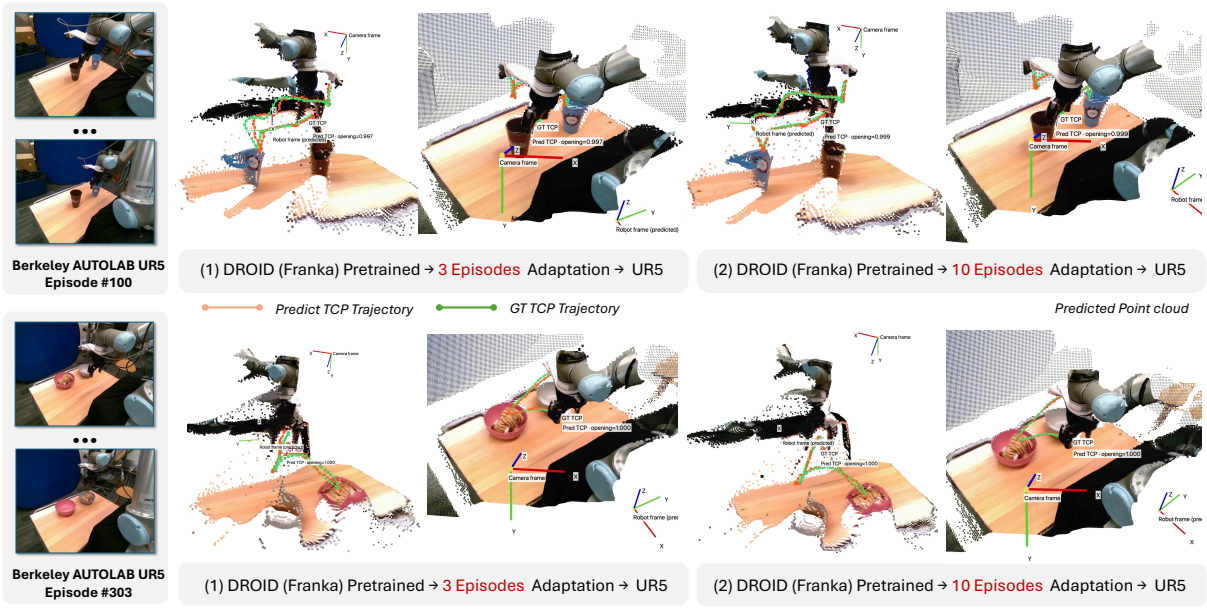

Figure 12: **Visualization of TCP4D for few-shot cross-embodiment adaptation.** We fine-tune TCP4D using only three and ten randomly selected episodes for three-shot and ten-shot transfer, respectively. The orange and green paths represent the predicted and ground-truth TCP trajectories, respectively. Our method can be readily transferred to new environments and robot embodiments.

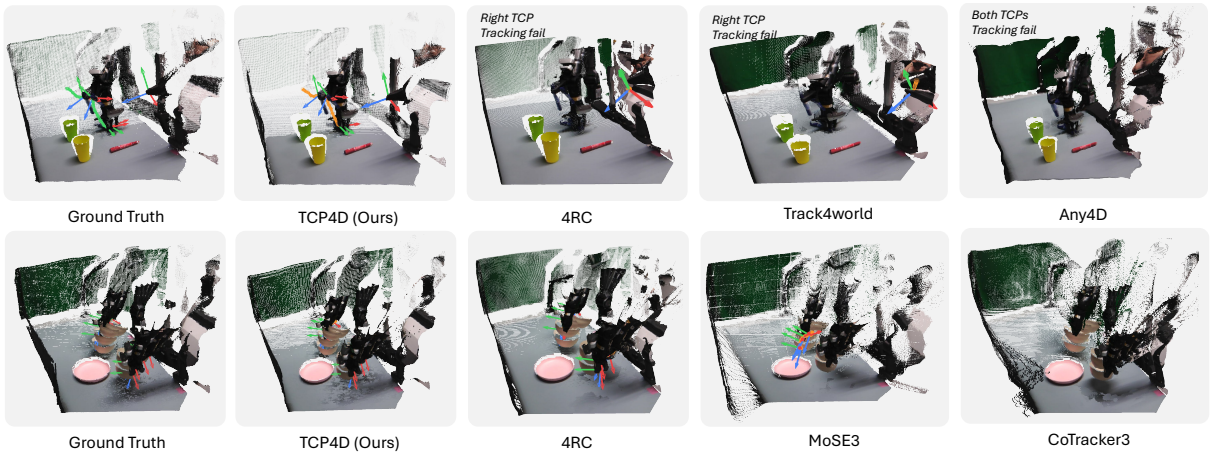

Figure 13: **Qualitative results of TCP4D Tracker on real-world task.** Methods such as 4RC and MOSE3 can experience tracking failures.

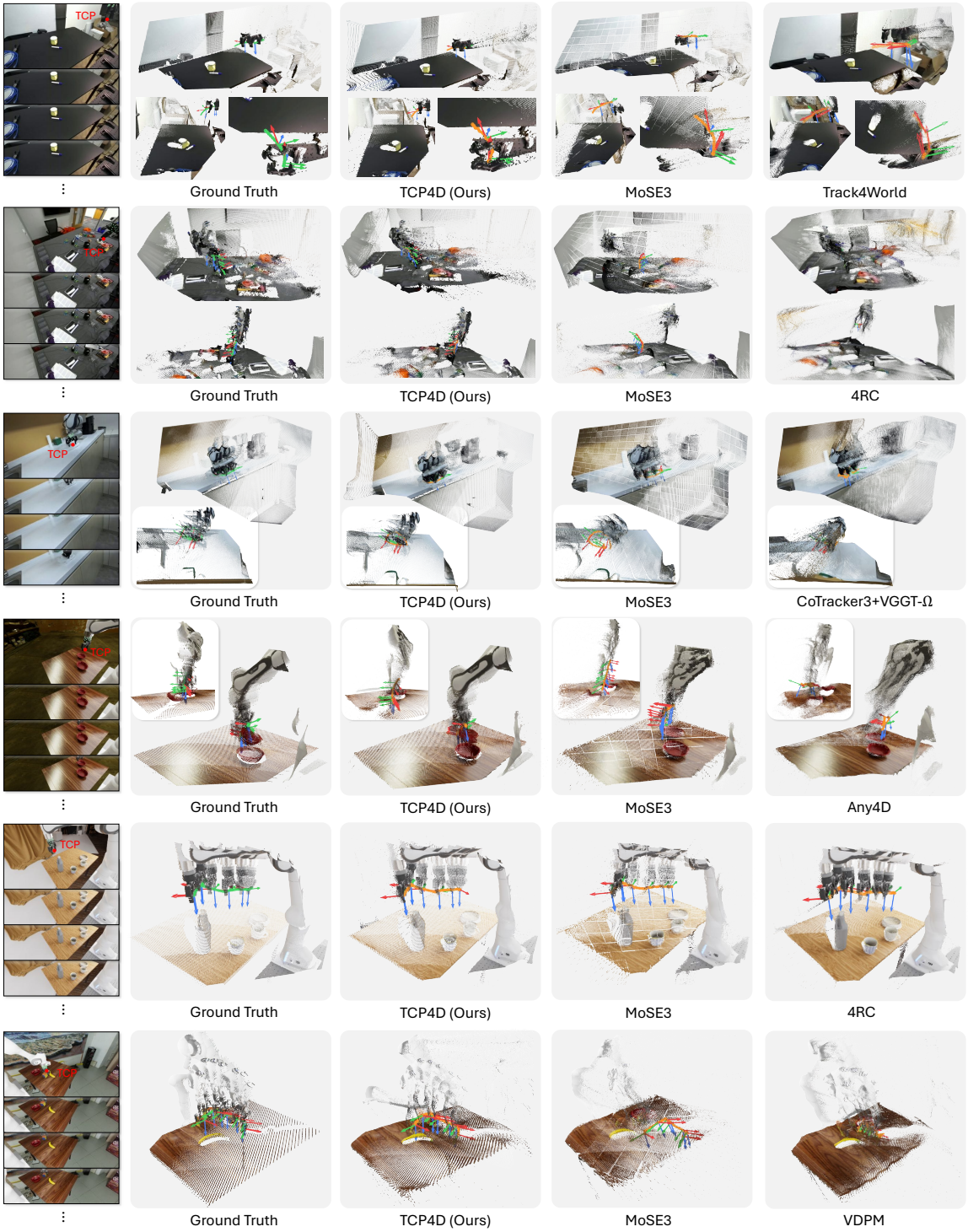

Figure 14: **Qualitative comparison of TCP tracking.** We compare TCP tracking by TCP4D against dense tracking baselines followed by post-processing. The left side shows the input video. The red point in the first frame marks the query used by TCP4D, while the green points indicate the rigid point group used by the baselines.

E Cross-Embodiment Data Training Details

E.1 Augmented Dataset

We augment 50 Aloha-AgileX demonstrations per task into ARX-X5 and UR5-WSG demonstrations across four RoboTwin tasks. For Place Burger Fries, Click Bell, and Handover Mic, we obtain 50 demonstrations for each target embodiment. For Shake Bottle, we obtain 49 ARX-X5 and 48 UR5-WSG demonstrations. This yields 397 augmented demonstrations from 400 source–target episode pairs, corresponding to a demonstration generation rate of 99.25% (Table 12). This rate measures successful demonstration generation among attempted source–target episode pairs, rather than downstream task success. Combining the source and augmented datasets gives 150 demonstrations per task, except Shake Bottle, which contains 147.

Table 12: **Augmented demonstration counts.** Generated demonstrations / attempted source–target episode pairs per target embodiment. Overall, 397/400 pairs yield augmented demonstrations (99.25%).

Task	**ARX-X5**	**UR5-WSG**
Click Bell	50/50	50/50
Shake Bottle	49/50	48/50
Handover Mic	50/50	50/50
Place Burger Fries	50/50	50/50
Total	**199/200**	**198/200**

E.2 Downstream Policy Training

We use Diffusion Policy with a randomly initialized ResNet18 visual encoder and a conditional 1D U-Net action denoiser. The policy receives three consecutive observations, each comprising a third-person RGB image and a 14-dimensional robot state. The state contains six joint positions and one continuous gripper value for each arm. Recorded source states are unnecessary for the RGB-based augmentation pipeline; downstream policies use the evaluated robot's joint and gripper states as proprioceptive inputs. Architecture and training hyperparameters are summarized in Table 13.

Table 13: **Downstream policy configuration.** Settings are shared across tasks and training-data regimes.

Parameter	**Value**
Policy	Diffusion Policy
Visual encoder	ResNet18, from scratch
Action denoiser	Conditional 1D U-Net
U-Net channel widths	256, 512, 1024
RGB observation	One camera, 240×320
State / action dimension	14 / 14
Observation / prediction horizon	3 / 8
Executed actions per prediction	6
Diffusion scheduler	DDPM (cosine noise schedule)
Diffusion / inference steps	100 / 100
State / action normalization	Min–max to $[-1, 1]$
Training epochs	600
Batch size	128
Optimizer	AdamW
Learning rate	10^{-4}
Weight decay	10^{-6}
AdamW betas	$(0.95, 0.999)$
Learning-rate schedule	Cosine
Warmup optimizer steps	500
Maximum EMA decay	0.9999
Training seed	0 (one run per setting)

All policies are trained from scratch, with separate models for different tasks. Within each task, the augmented-only setting uses independent ARX-X5 and UR5-WSG models, whereas the source-plus-augmented setting uses one shared model across Aloha-AgileX, ARX-X5, and UR5-WSG. For joint training, demonstrations from the three embodiments are concatenated, and valid observation–action sequences are sampled uniformly without explicit embodiment balancing. All training settings use the same number of epochs; consequently, the number of optimization updates varies with dataset size.

For each task and evaluated embodiment, policy evaluation uses 50 episodes following a common environment-seed schedule across training settings. Success is determined by RoboTwin's task-specific criteria, and the success rate is the fraction of evaluation episodes completed successfully. Results are obtained from one training seed per setting. Downstream results are reported in Fig. 6 and discussed in Sec. 4.3.

Qualitative Results. Fig. 15 illustrates the rendered cross-embodiment demonstrations. These examples show the visual output of retargeting and composition; downstream policy performance is evaluated separately in Fig. 6.

F Manipulation Evaluation

F.1 TCP4D Generator Result on RoboTwin

Dataset. We collect 50 clean demonstrations for each of 48 tasks, totaling 2,400 training episodes, with third-person and left & right wrist images as the scene observations. We exclude the *Dump_Bin_BigBin* and *Put_Object_Cabinet* tasks because the TCP is totally occluded in the third-person camera view. RoboTwin is sampled at 16.67 Hz.

Results. In *clean2clean*, policies are evaluated in the clean environment; in *clean2random*, they are evaluated in the randomized environment. The latter setting tests generalization from clean demonstrations to environmental randomization.

Table 14 reports per-task success rates of TCP4D for all 48 RoboTwin 2.0 tasks under *clean2clean* and *clean2random*.

Table 14: **Per-task success rates (%) of TCP4D on RoboTwin 2.0.** The 48 tasks match Table 3. E and H denote Easy (*clean2clean*) and Hard (*clean2random*), respectively. Average reports the mean over all 48 tasks.

Task	E	H	Task	E	H	Task	E	H
Adjust Bottle	100	2	Open Microwave	77	3	Place Object Scale	33	1
Beat Block Hammer	82	1	Pick Diverse Bottles	86	8	Place Object Stand	79	12
Blocks Ranking RGB	63	1	Pick Dual Bottles	97	8	Place Phone Stand	88	2
Blocks Ranking Size	27	0	Place A2B Left	43	5	Place Shoe	56	4
Click Alarmclock	100	52	Place A2B Right	36	1	Press Stapler	89	41
Click Bell	97	41	Place Bread Basket	84	8	Put Bottles Dustbin	81	46
Grab Roller	100	22	Place Bread Skillet	75	3	Rotate QRcode	72	0
Handover Block	85	8	Place Burger Fries	93	15	Scan Object	67	2
Handover Mic	100	0	Place Can Basket	52	2	Shake Bottle Horizontally	99	81
Hanging Mug	33	0	Place Cans Plasticbox	89	3	Shake Bottle	97	72
Lift Pot	44	0	Place Container Plate	95	15	Stack Blocks Three	50	0
Move Can Pot	47	4	Place Dual Shoes	43	0	Stack Blocks Two	88	1
Move Pillbottle Pad	37	1	Place Empty Cup	100	0	Stack Bowls Three	54	3
Move Playingcard Away	85	6	Place Fan	24	1	Stack Bowls Two	86	9
Move Stapler Pad	31	0	Place Mouse Pad	26	0	Stamp Seal	12	0
Open Laptop	88	8	Place Object Basket	62	4	Turn Switch	69	29
Average over all 48 tasks							**69.19**	**10.94**

F.2 Real-world Task Performance

We evaluate two tasks: *1) placing pens* and *2) stacking bowls* with the left and right arms. We collect 60 demonstration trajectories per task for training. We used LingBot-Depth (Tan et al., 2026) to process the raw depth data captured by the sensor.

Figure 15: **Qualitative results of our cross-embodiment augmented data.** Given an RGB-only video, TCP4D reconstructs the TCP's 3D trajectory and gripper state, enabling augmentation on any embodiment that satisfies *reachability*. The right side shows the resulting augmented trajectories rendered on different target embodiments.

Qualitative results of TCP4D Tracker on real-world tasks. We provide a visualization of TCP4D Tracker on real-world tasks in Fig. 13.

Task Rubric. The task-specific progress scoring rubric used in Fig. 8 is summarized in Table 15.

Real-world Task visualizations. We provide complete rollout visualizations for both tasks in Fig. 16 and Fig. 17

Table 15: **Real-world task progress-score rubric.** Parentheses indicate the cumulative score after completing each ordered subtask.

Task	Ordered subtasks
Placing_Pens ($R = 4$)	**(1/4)** right arm picks up the pen on the right side of the table; **(2/4)** right arm places the pen in the right pen holder; **(3/4)** left arm picks up the pen on the left side of the table; **(4/4)** left arm places the pen in the left pen holder.
Stacking_bowls ($R = 4$)	**(2/4)** both arms simultaneously pick up two bowls; **(3/4)** right arm places its bowl on the plate; **(4/4)** left arm stacks its bowl on top of the first bowl.

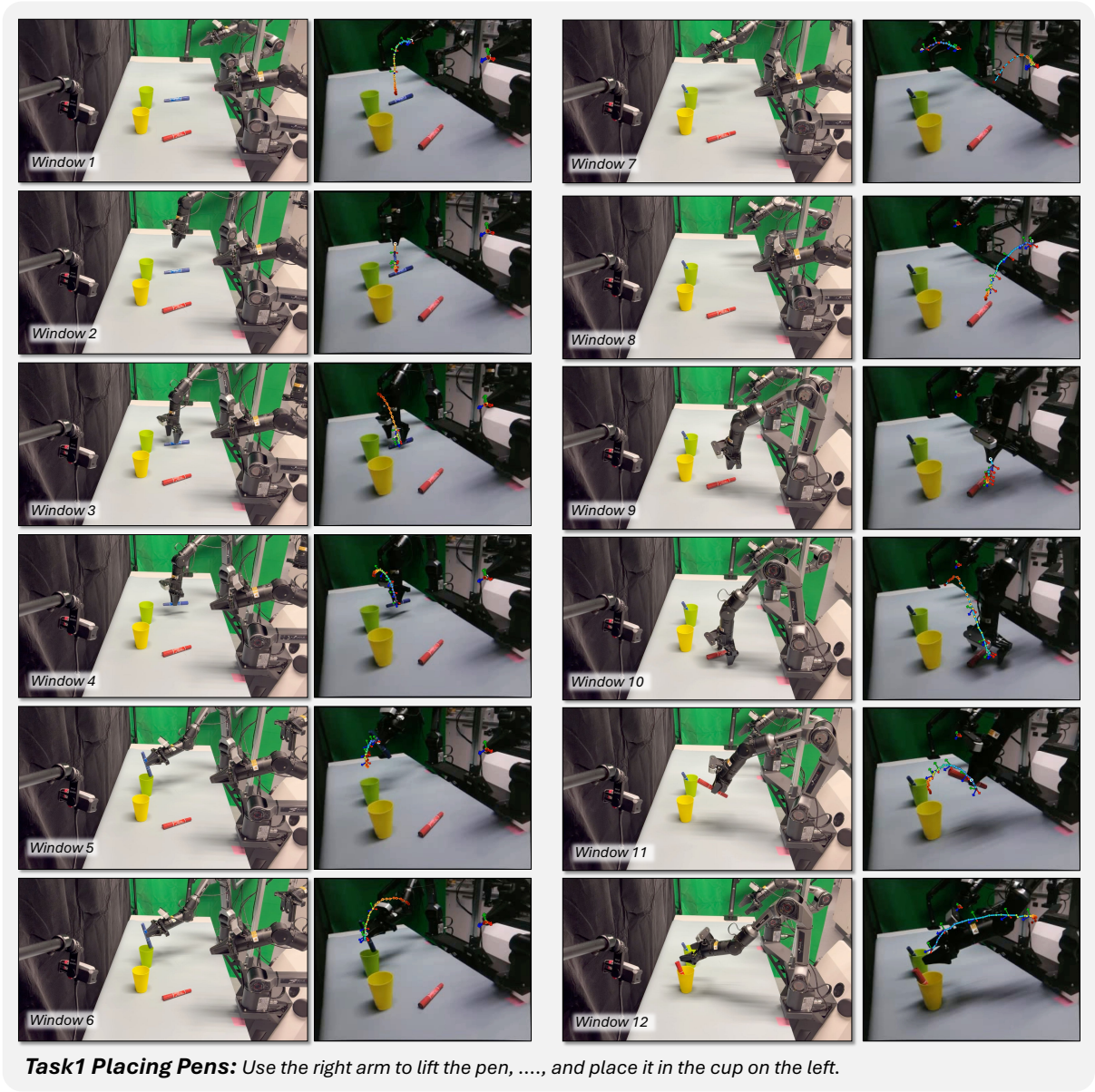

Figure 16: **Visualizations of TCP4D on Task *Placing Pens*.** Each window shows the future TCP trajectory predicted by TCP4D. The motion is then planned using the cuRobo (Sundaralingam et al., 2026) library.

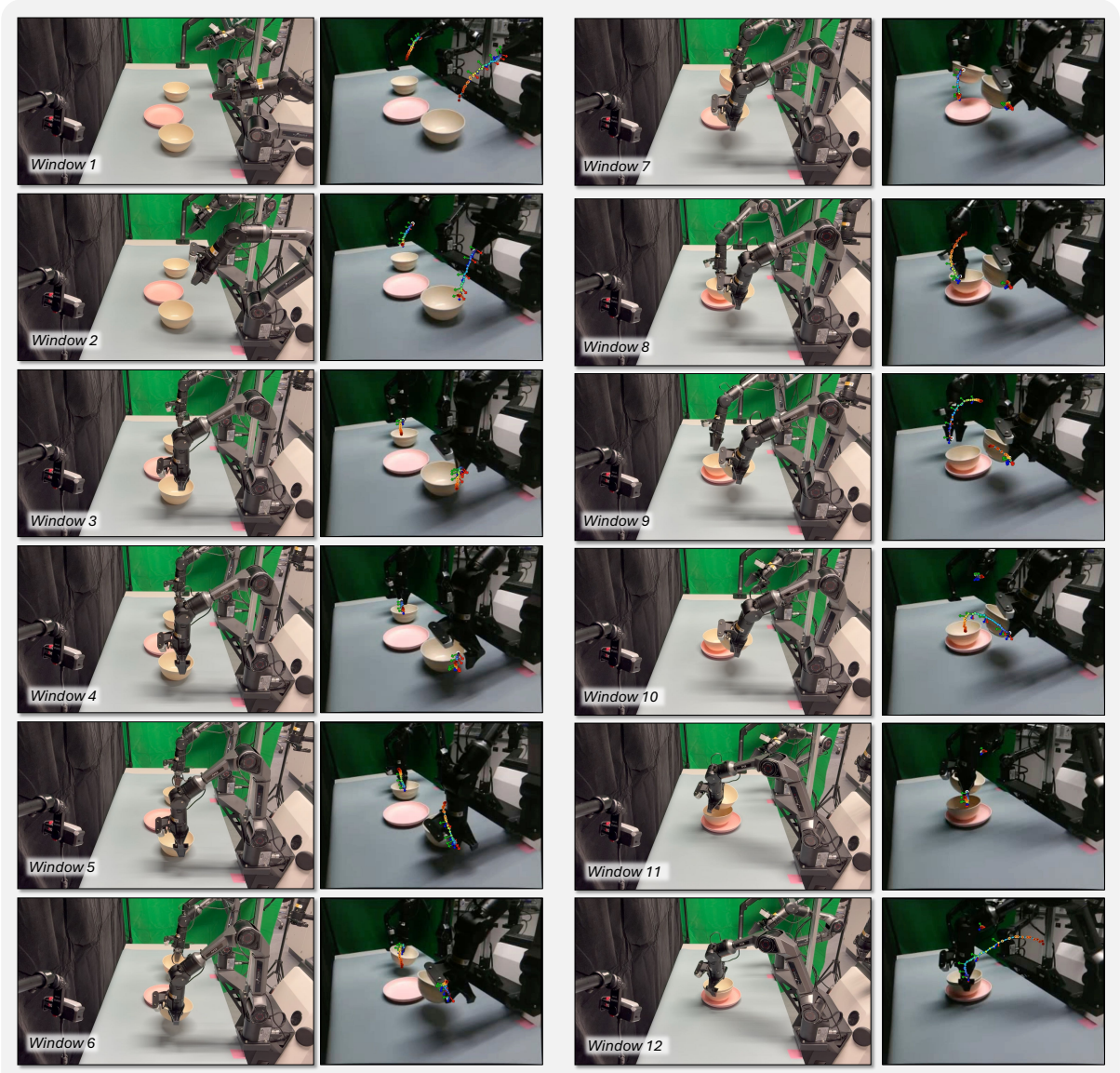

Figure 17: **Visualizations of TCP4D on Task *Stack Bowls*.** Each window shows the future TCP trajectory predicted by TCP4D. The motion is then planned using the cuRobo (Sundaralingam et al., 2026) library.

G Limitations and Future Work

Embodiment Coverage. While TCP4D demonstrates strong performance in DROID setting, its training data currently covers only embodiments with Franka and Robotiq grippers, and other embodiment data remains limited. After pretraining on DOIRD data, the TCP4D adapts effectively to UR5-WSG with only a few demonstrations (Table 2). These results support the use of a shared $\mathrm{SE}(3)$ TCP representation for cross-embodiment transfer. With sufficient training data from a diverse range of embodiments, we expect TCP4D to generalize well to new robot platforms and gripper designs.

Environmental Generalization. The TCP4D Generator shows strong performance under RoboTwin *clean2clean*, but its performance under *clean2random* remains limited. The Generator is trained on a relatively small set of clean demonstrations without large-scale policy pretraining, providing little exposure to the environmental variations encountered during randomized evaluation. This limited training diversity may partly explain the performance gap. Large-scale pretraining on diverse manipulation data, together with domain randomization and targeted data augmentation, offers practical directions for addressing this gap and realizing the broader potential of 4D-conditioned TCP generation.